

CSC-648-01-SU26

Project: Student Center

Team #4: BigBacks

Version History

Milestone 2 Version 1	Date: 6/17/2026
Milestone 2 Version 2	Date: 7/1/2026

Team Members

Wen Chien Yen	Team Lead / Scrum Master	wyen@sfsu.edu
Oscar Wang	Frontend Lead / Technical Writer	owang1@sfsu.edu
Banghao Yuan	Software Architect	Byuan1@sfsu.edu
Sean Wang	Database Administrator	swang29@sfsu.edu
Enoch Lin	Github Master	elin5@sfsu.edu
Kashif Zada	Backend Lead	kzada@sfsu.edu

Revision History

Date	Member	Task	Details
7/1/2026	Banghao Yuan	Backend Architecture	Adding describe the incremental software process and explain the application and justification of creational and structural design patterns. Updated the High Level APIs and Algorithms section to emphasize major application services and non-trivial algorithms instead of individual REST endpoints.
6/29/2026	Sean Wang	Database Architecture	Created seed.sql to populate several tables in the database (entities related to course search). Connected frontend with backend by creating a course search page using courses.js from Banghao. The Course Search utilizes ORDER BY sorting algorithm to order course items.
6/29/2026	Banghao Yuan	Backend Architecture	Connected the backend with the database, prepared API endpoints for frontend-backend communication, and implemented the functional requirements for user registration and course search.

6/28/2026	Banghao Yuan	Backend Architecture	Created the initial UML diagram. Mapped frontend modules to REST API endpoints, backend controllers, PostgreSQL/RDS database entities, and data flow. Drafted the first version of the API design and supported frontend/backend architecture planning.
6/28/2026	Sean Wang	Database Architecture	Finalized list of functional requirements for M2v1. Updated ERD with cardinality.
6/28/2026	Wen-Chien Yen	Network and Deployment Design	Added current AWS deployment and explanation, including EC2 Ubuntu server, Nginx, React/Vite frontend deployment, GitHub Actions SSH deployment flow, privates Amazon RDS PostgreSQL access, and instructor verification tunnel.
6/28/2026	Wen-Chien Yen	Project Management	Added the project management section explaining how Milestone 2 work as divided, how team members managed assigned responsibilities, how GitHub and shared documentation were used and how future milestones will be managed through featured-based implementation and

			integration testing.
6/28/2026	Oscar Wang	Frontend Structure	Created the skeleton base for the way the website is navigated, as well as restructured a few frontend entities to be more clean code-wise. Also added a template for the login page.
6/27/2026	Sean Wang	Database Architecture	Created PostgreSQL schema based on M2v1 design. Added ERD based on database design.
6/26/2026	Banghao Yuan	Data Definition and Prioritized High Level Functional Requirements	Added data definitions for core Student Center entities, including users, students, courses, class sections, schedules, campus map data, notifications, and financial records. Clarified key attributes and relationships for each entity. Organized and refined the high-level functional requirements for the Student Center system. Grouped requirements based on M1v2 into Priority 1, Priority 2, Priority 3, Maybe Later, and Out of Scope categories to clarify the current prototype scope and future development priorities.
6/24/2026	Kashif Zada	Key project risks, external dependencies, and	Wrote the Key Project Risks section covering external

		security discussion	system dependencies, degree progress logic, database schema stability, security, team integration, and scope/time constraints. Documented External Dependencies (AWS EC2, RDS PostgreSQL, Nginx, GitHub Actions, SSO, Canvas, financial aid and payment portals) and the Security Discussion (SSH key access, security groups, role-based access control, password hashing, HTTPS, and secrets management).
--	--	---------------------	---

Table of Contents

Revision History.....	2
Data Definitions.....	8
User and Role.....	8
Academic Data.....	9
Enrollment and Class data.....	11
Schedule and Planning Data.....	12
Campus Navigation Data.....	14
Financial Data.....	15
Prioritized High Level Functional Requirements.....	17
Priority 1.....	17
Authentication / User.....	17
Student Profile.....	17
Course Search.....	17
Class Section.....	17
Planned Schedule.....	17
Semester.....	18
Department.....	18
Priority 2.....	18
Campus Map.....	18
Calendar.....	18
Cart.....	19
Degree Progress Tree.....	19
Admin Management.....	20
Priority 3.....	20
Enrollment Information and Enrollment Records.....	20
Notifications.....	21
Financial Data.....	21
Maybe Later.....	21
Out of Scope for Current Version.....	22
Mockups and Storyboards.....	23
High Level System Design.....	34
Database Architecture.....	34
Functional Requirements.....	34
Entity Descriptions.....	40
DBMS Selection.....	41
ERD (Entity-Relationship Diagram).....	42
Media Storage Strategy.....	42
Backend Architecture.....	44
UML Diagram.....	47
Network and Deployment Design.....	49
Current Network and Deployment infrastructure.....	50

Deployment Components.....	51
Network Protocols and Ports.....	51
Security Notes.....	52
High Level APIs and Algorithms.....	53
Internal Backend APIs.....	53
External Libraries / APIs.....	56
Key Project Risks.....	58
1. External System Dependencies.....	58
2. Degree Progress Tree and Prerequisite Logic.....	58
3. Database Schema Stability.....	58
4. Security.....	58
5. Team Integration.....	58
6. Scope and Time Constraints.....	59
External Dependencies.....	59
Security Discussion.....	60
Project Management.....	61
Team Contributions.....	63
Team Contributions.....	63
Wen-Chien Yen.....	63
Oscar Wang.....	63
Banghao Yuan.....	64
Sean Wang.....	64
Enoch Lin.....	64
Kashif Zada.....	65

Data Definitions

Our Student Center application uses structured academic, Planning ,campus navigation, and financial data to support student center academic planning.

User and Role

1. User

- A User is any individual who can access the Student Center system. A User stores both personal information and account information used for authentication and role-based access.
- Attributes:
 - user_id: Unique identifier for the user.
 - first_name: User's first name.
 - last_name: User's last name.
 - email: User's university email address.
 - phone_number: Optional contact phone number.
 - created_date: Date when the user record was created.
- Relationship: A User has one User Account. A User may act as a Student, or Admin depending on the assigned role.

2. Student

- A student is a User who has access to academic planning tools, including course search, planned schedule, calendar, degree progress, campus map, car, notifications, and financial information.
- Attributes:
 - student_id: Unique identifier for the student.
 - account_id: Reference to the student's User Account.
 - major_id: Reference to the student's declared Major.
 - academic_level: Student level, such as freshman, sophomore, junior, senior, or graduate.
 - student_type: Student category, such as first-time, continuing, transfer, or international.
 - catalog_year: Academic catalog year used for degree requirement rules.
- Relationship: A student may have many Planned Schedules. Cart records Enrollment records, Notification and Financial records. A student belongs to one declared Major for degree planning purposes.

3. Admin

- An Admin is a User who has permission to manage academic and system data, including courses, class sections, buildings, classrooms, degree requirements, users, and student financial information.
- Attributes
 - admin_id: Unique identifier for the admin.
 - account_id: Reference to the admin's User Account.
 - permission_level: Level or category of administrative access.

- department_scope: Optional department or academic unit the admin manages.
 - Relationships: Admin users should only access admin functions through role-based authorization. Admin actions may affect Course, Class Section, Building, Classroom, Degree Requirement, and User data.
- 4. Profile (UI-Generated Feature)**
- A Profile is the user-facing display of a User's personal and academic information. It is not necessarily a separate database table; it may be generated from User, User Account, Student, and Major data.
 - Attribute:
 - display_name: User's full name.
 - email: University email address.
 - role: User role shown in the interface.
 - academic_level: Student academic level, if applicable.
 - student_type: Student type, if applicable.
 - declared_major: Student's declared major, if applicable.
 - contact_information: Editable phone or contact details.
 - Relationships: The Profile page should use data from the User, User Account, Student,, and Major records.
- 5. Notification**
- A Notification is a system-generated message that informs a User about important academic or system events.
 - Attribute
 - notification_id: Unique identifier for the notification.
 - user_id: Reference to the User receiving the notification.
 - notification_type: Type of notification, such as deadline, conflict, success, warning, or system message.
 - message: Notification text displayed to the user.
 - created_date: Date and time when the notification was created.
 - is_read: Indicates whether the user has read the notification.
 - dismissed_date: Date and time when the user dismissed the notification, if applicable.
 - Notifications may be generated by enrollment periods, approaching deadlines, schedule conflicts, insufficient walking time, or successful and failed system actions.

Academic Data

- 6. Department**
- A Department is an academic unit that offers Courses, manages Majors, and includes Instructor name.
 - Attributes
 - department_id: Unique identifier for the department.
 - department_name: Official department name.

- office_location: Department office location.
- contact_email: Department contact email.
- phone_number: Department phone number.
- Relationship: A Department may offer many Courses, contain many Majors, and have many Instructors.

7. Major

- A Major is a student's declared academic program. It defines the degree requirements used by the Degree Progress Tree.
- Attributes
 - major_id: Unique identifier for the major.
 - department_id: Reference to the Department that owns the major.
 - major_name: Official major name.
 - required_units: Total units required for the major or degree.
 - catalog_year: Catalog year for the major requirements.
 - degree_type: Degree type, such as B.S., B.A., M.S., or M.A.
- Relationship: A Major belongs to one Department and contains many Degree Requirements. A Student declares one Major for degree planning purposes.

8. Degree Requirement

- A Degree Requirement is a requirement that a Student must complete for graduation. Requirements may include major required courses, major electives, general education categories, unit requirements, and prerequisite rules.
- Attributes:
 - requirement_id: Unique identifier for the requirement.
 - major_id: Reference to the related Major.
 - requirement_category: Requirement type, such as major core, major elective, GE, prerequisite, or total units.
 - required_units: Number of units required.
 - requirement_description: Human-readable explanation of the requirement.
 - requirement_status: Status such as completed, planned, or remaining.
 - minimum_grade: Minimum grade needed to satisfy the requirement, if applicable.
- Relationship: A Degree Requirement may be satisfied by one or more Courses. The Degree Progress Tree uses Degree Requirement data to show completed, planned, and remaining requirements

9. Course

- A Course is an academic subject offered by a Department. It describes the general course information and does not represent a specific scheduled class meeting.
- Attribute:
 - course_id: Unique identifier for the course.
 - department_id: Reference to the Department offering the course.
 - course_code: Official course code, such as CSC 648.
 - course_title: Course title.
 - units: Number of academic units.
 - description: Course description.

- course_level: Level such as lower-division, upper-division, or graduate.
- course_attributes: Attributes such as GE area, lecture, lab, or repeatable status.
- prerequisites: Required courses or conditions before taking this course.
- Relationship: A Course may have many Class Sections across different Semesters. A Course may satisfy one or more Degree Requirements.

Enrollment and Class data

10. Class Section

- A Class Section is a specific scheduled offering of a Course in a Semester. It includes instructor name, meeting time, classroom, seat availability, and enrollment status.
- Attribute
 - section_id: Unique identifier for the class section.
 - course_id: Reference to the related Course.
 - semester_id: Reference to the Semester when the section is offered.
 - instructor_name: name of the Instructor teaching the section.
 - classroom_id: Reference to the assigned Classroom.
 - section_number: Official section number.
 - meeting_days: Days when the section meets.
 - start_time: Class start time.
 - end_time: Class end time.
 - modality: In-person, online, or hybrid.
 - meeting_type: Synchronous or asynchronous.
 - seat_capacity: Maximum number of students allowed.
 - seats_available: Number of remaining available seats.
 - waitlist_capacity: Maximum waitlist size.
 - waitlist_available: Number of remaining waitlist spots.
 - enrollment_status: Open, closed, waitlisted, or cancelled.
- A Class Section belongs to one Course and one Semester. It may appear in a Student's Planned Schedule, Cart, Calendar, or Enrollment Record.

11. Semester

- A Semester is an academic term in which Class Sections are offered and enrollment activity occurs.
- Attribute:
 - semester_id: Unique identifier for the semester.
 - term: Term name, such as Spring, Summer, or Fall.
 - year: Calendar year of the term.
 - start_date: Semester start date.
 - end_date: Semester end date.
 - add_drop_deadline: Last date for add/drop actions.
 - withdrawal_deadline: Last date for withdrawal.
 - enrollment_start_date: Date when enrollment begins.
 - enrollment_end_date: Date when enrollment ends.
- Relationship: A Semester contains many Class Sections. Planned Schedules, Carts, and Enrollment Records are associated with a Semester.

12. Enrollment Information

- Enrollment Information describes when and how a Student is allowed to enroll, add, drop, or withdraw from Class Sections for a specific Semester.
- Attributes:
 - enrollment_info_id: Unique identifier for enrollment information.
 - semester_id: Reference to the related Semester.
 - student_type: Student type affected by the enrollment window.
 - enrollment_start_time: Start time for enrollment access.
 - enrollment_end_time: End time for enrollment access.
 - add_drop_deadline: Deadline for adding or dropping classes.
 - withdrawal_deadline: Deadline for withdrawing from classes.
- Relationships: Enrollment Information is used to determine whether a Student can perform enrollment actions during a given term.

13. Enrollment Record

- An Enrollment Record stores a Student's actual enrollment history or current enrollment in a Class Section.
- Attributes:
 - enrollment_id: Unique identifier for the enrollment record.
 - student_id: Reference to the Student.
 - section_id: Reference to the Class Section.
 - semester_id: Reference to the Semester.
 - enrollment_status: Status such as enrolled, dropped, withdrawn, completed, or waitlisted.
 - enrolled_date: Date when the student enrolled.
 - dropped_date: Date when the student dropped the section, if applicable.
 - grade: Final grade, if available.
- Relationships: Enrollment Records represent official or sample academic history. They are different from Planned Schedule and Cart data because they represent actual enrollment rather than planning intent.

Schedule and Planning Data

14. Planned Schedule

- A Planned Schedule is a collection of Class Sections that a Student is considering or planning to take for a Semester.
- Attribute:
 - planned_schedule_id: Unique identifier for the planned schedule.
 - student_id: Reference to the Student who owns the planned schedule.
 - semester_id: Reference to the Semester being planned.
 - created_date: Date when the planned schedule was created.
 - last_updated: Date when the planned schedule was last modified.
 - total_units: Total units from selected Class Sections.
 - schedule_status: Status such as draft, active, or archived.
- Relationships :A Planned Schedule may contain many Class Sections. It supports time conflict detection, prerequisite checking, walking time calculation, and Calendar display. The Planned Schedule may also support

basic walking time checking between selected Class Sections when building information is available.

○

15. Calendar (UI-Generated Feature)

- A Calendar is a weekly or monthly view of a Student's planned and enrolled Class Sections. It is primarily a user interface view generated from Planned Schedule and Enrollment Record data.
- Attributes:
 - `calendar_view_type`: Weekly or monthly view.
 - `student_id`: Reference to the Student.
 - `semester_id`: Reference to the Semester.
 - `displayed_sections`: Planned and enrolled Class Sections shown on the calendar.
 - `selected_date_range`: Date range being displayed.
- Relationships: The Calendar should display course code, meeting time,, building, and room. It should visually distinguish planned classes from enrolled classes.

16. Cart

- A Cart is a collection of Class Sections selected by a Student before final enrollment. It allows the Student to review selected sections, total units, enrollment status, and possible conflicts before enrolling.
- Attributes:
 - `cart_id`: Unique identifier for the cart.
 - `student_id`: Reference to the Student who owns the cart.
 - `semester_id`: Reference to the Semester.
 - `created_date`: Date when the cart was created.
 - `last_updated`: Date when the cart was last modified.
 - `total_units`: Total units in the cart.
 - `cart_status`: Status such as active, submitted, or archived.
- Relationships: A Cart may contain many Class Sections. A Cart is different from a Planned Schedule because it is closer to enrollment action.

17. Degree Progress Tree (UI-Generated Feature)

- A Degree Progress Tree is a visual academic roadmap that shows a Student's completed, planned, and remaining Degree Requirements.
- Attributes:
 - `student_id`: Reference to the Student.
 - `major_id`: Reference to the Student's declared Major.
 - `completed_requirements`: Requirements already satisfied by completed courses.
 - `planned_requirements`: Requirements expected to be satisfied by planned courses.
 - `remaining_requirements`: Requirements not yet satisfied.
 - `remaining_units`: Units still needed for graduation.
 - `progress_percentage`: Estimated percentage of degree completion.
- Relationships: The Degree Progress Tree is generated from Student, Major, Degree Requirement, Course, Enrollment Record, and Planned Schedule data.

It helps Students understand how selected and completed courses connect to graduation requirements.

18. Course Details (UI-Generated Feature)

- Course Details is a user-facing view that combines Course information with available Class Sections, instructor information, schedule information, location, prerequisites, and seat availability.
- Attributes:
 - `course_id`: Reference to the Course.
 - `course_code`: Official course code.
 - `course_title`: Course title.
 - `description`: Course description.
 - `units`: Course units.
 - `prerequisites`: Required courses or conditions.
 - `available_sections`: List of related Class Sections.
 - `instructor_information`: Instructor name and office hour details.
 - `location_information`: Building and classroom information.
 - `seat_availability`: Seats available for each section.
- Relationships: Course Details may not need to be a separate database table. It can be generated by joining Course, Class Section, Classroom, Building, and Semester data.

19. Course Suggestion

- A Course Suggestion is a planning recommendation for Students who are close to graduation and may want to take advanced courses for graduate school preparation or career development.
- Attributes:
 - `suggestion_id`: Unique identifier for the suggestion.
 - `student_id`: Reference to the Student receiving the suggestion.
 - `course_id`: Reference to the suggested Course.
 - `reason`: Explanation of why the course is suggested.
 - `suggestion_type`: Type such as graduate preparation, career preparation, or major elective.
 - `confidence_level`: Optional value showing strength of the suggestion.
 - `created_date`: Date when the suggestion was generated.
- Relationships: Course Suggestions are planning aids only. They should not be presented as official academic advising or guaranteed graduate credit approval.

Campus Navigation Data

20. Campus Map (UI-Generated Feature)

- A Campus Map is a map-based interface that displays campus buildings, classroom locations, department locations, and campus resources.
- Attribute:
 - `map_name`: Name of the campus map.
 - `displayed_buildings`: Buildings shown on the map.
 - `displayed_resources`: Campus resources shown on the map.

- Relationships: The Campus Map is mainly a user interface feature. It uses Building and Classroom data to highlight locations for selected Class Sections.

21. Building

- A Building is a physical campus structure where classes, departments, offices, or resources may be located.
- Attributes:
 - building_id: Unique identifier for the building.
 - building_name: Official building name.
 - address: Optional campus address or description.
- Relationships: A Building may contain many Classrooms and may be displayed on the Campus Map.

22. Classroom

- A Classroom is a room inside a Building where a Class Section may meet.
- Attributes:
 - classroom_id: Unique identifier for the classroom.
 - building_id: Reference to the Building.
 - room_number: Official room number.
 - room_capacity: Number of seats available in the room.
 - classroom_type: Lecture room, lab, seminar room, or other type.
- Relationships: A Classroom belongs to one Building. A Class Section may be assigned to one Classroom

Financial Data

23. Billing Profile

- A Billing Profile is a Student's financial account profile in the system. It connects the Student to charges, payments, refunds, financial aid credits, and outstanding balance information.
- Attribute:
 - billing_profile_id: Unique identifier for the billing profile.
 - student_id: Reference to the Student.
 - billing_status: Current billing status.
 - billing_email: Email used for billing contact.
 - billing_phone: Phone number used for billing contact.
 - last_updated: Date when billing information was last updated.
 - current_balance: Current outstanding balance.
- Relationships: A Billing Profile may have many Student Charges, Transactions, and Financial Aid records.

24. Student Charge

- A Student Charge is a financial charge assigned to a Student for a specific Semester or service.
- Attribute:
 - charge_id: Unique identifier for the charge.
 - billing_profile_id: Reference to the Billing Profile.
 - semester_id: Reference to the related Semester, if applicable.
 - charge_type: Type of charge, such as tuition, fee, housing, or other.
 - amount: Charge amount.

- due_date: Payment due date.
- payment_status: Status such as unpaid, partially paid, paid, or overdue.
- Relationships: Student Charges contribute to the Billing Profile's outstanding balance.

25. Financial Aid

- Financial Aid is aid information associated with a Student, such as loans, grants, scholarships, or awards.
- Attributes:
 - financial_aid_id: Unique identifier for the financial aid record.
 - student_id: Reference to the Student.
 - billing_profile_id: Reference to the Billing Profile.
 - aid_name: Name of the aid.
 - aid_type: Type such as loan, grant, scholarship, or award.
 - semester_id: Semester when the aid applies.
 - amount: Aid amount.
 - aid_status: Status such as pending, accepted, declined, or applied.
 - application_link: Link to the official financial aid application website, if applicable.
- Relationships: Financial Aid may reduce the Student's outstanding balance when applied. Financial aid application links should redirect students to official university or external financial aid systems when appropriate

26. Transaction

- A Transaction is a financial activity related to a Student's Billing Profile, such as a payment, refund, charge adjustment, or financial aid credit.
- Attributes:
 - transaction_id: Unique identifier for the transaction.
 - billing_profile_id: Reference to the Billing Profile.
 - transaction_type: Payment, refund, charge, or financial aid credit.
 - amount: Transaction amount.
 - transaction_date: Date of the transaction.
 - transaction_status: Status such as pending, completed, failed, or cancelled.
 - description: Description of the transaction.
 -
- Relationships: Transactions affect the current balance shown in the Billing Profile.

Prioritized High Level Functional Requirements

Priority 1

Authentication / User

- FR 1.1: A User shall be able to log in with an email and password for the prototype.(maybe change to username or userID)
- FR 1.2: A User shall be able to log out of their account.
- FR 1.3: A User shall be identified as either a Student or an Admin.
- FR 1.4: A User shall be shown features based on their assigned role.

Student Profile

- FR 2.1: A Student shall have an academic level, such as freshman, sophomore, junior, senior, or graduate.
- FR 2.2: A Student shall have a student type, such as first-time student, continuing student, transfer student, or international student.
- FR 2.3: A Student shall be able to view and update their profile.

Course Search

- FR 2.4: A Student shall be able to search for courses.
- FR 2.5: A Student shall be able to filter courses.
- FR 2.6: A Student shall be able to view course details.
- FR 5.1: A Course shall have basic course information, including course code, title, and units.
- FR 5.2: A Course shall be associated with a Department.
- FR 5.3: A Course shall have a detailed course description.
- FR 5.7: A Course shall have available Class Sections for a selected term.
- FR 5.8: A Course shall be searchable through class search.
- FR 5.9: A Course shall be filterable through class search.

Class Section

- FR 6.1: A Class Section shall have section details, including section number, term, instructor name, meeting time, location, enrollment status, and seat availability.
- FR 6.2: A Class Section shall be addable or removable from a Student's Planned Schedule.
- FR 6.3: A Class Section shall have a modality, such as in-person, online, or hybrid.
- FR 6.4: A Class Section shall have a meeting type, such as synchronous or asynchronous.
- FR 6.7: A Class Section shall indicate whether it conflicts with Class Sections already in a Student's Planned Schedule.

Planned Schedule

- FR 2.8: A Student shall be able to add or remove courses from a Planned Schedule.
- FR 2.11: A Student shall be able to view their Planned Schedule.
- FR 7.1: A Planned Schedule shall store all Class Sections selected by a Student.
- FR 7.2: A Planned Schedule shall display the relevant course information for each selected Class Section, including course code, section number, meeting time, location, modality, instructor name, and enrollment status.
- FR 7.3: A Planned Schedule shall allow a Student to add or remove Class Sections.
- FR 7.4: A Planned Schedule shall detect time conflicts between selected Class Sections.

Semester

- FR 13.1: A Semester shall show term name, start date, end date, add/drop deadline, withdrawal deadline, and enrollment period.

Department

- FR 14.1: A Department shall have a department name, office location, and contact information.
 - FR 14.2: A Department shall show a list of Courses.
-

Priority 2

Campus Map

- FR 2.16: A Student shall be able to view class building locations on the Campus Map.
- FR 8.1: A Campus Map shall display campus building locations.
- FR 8.2: A Campus Map shall display the building location for a selected Class Section.
- FR 8.3: A Campus Map shall display classroom location information for a selected Class Section.
- FR 8.4: A Campus Map shall display campus resources, such as library, gym, student center, HSS buildin...
- FR 8.6: A Campus Map shall allow a Student to return to the Calendar after viewing a class location.

Calendar

- FR 20.1: A Calendar shall display a Student's planned and enrolled Class Sections in a weekly or monthly format.
- FR 20.2: A Calendar shall display each Class Section on the correct day and time.
- FR 20.3: A Calendar shall visually distinguish planned classes from enrolled classes.
- FR 20.4: A Calendar shall display relevant course information, including course code, meeting time, instructor name, building, and room.

- FR 20.5: A Calendar shall allow a Student to select a Class Section and view its Course Details.
- FR 20.6: A Calendar shall allow a Student to select a class location and open it on the Campus Map.

Cart

- FR 2.24: A Student shall be able to view their Cart.
- FR 2.25: A Student shall be able to add or remove Class Sections from their Cart.
- FR 6.5: A Class Section shall be addable to a Student's Cart.
- FR 6.6: A Class Section shall indicate whether it is eligible to be added to a Cart.
- FR 21.1: A Cart shall store Class Sections selected by a Student.
- FR 21.2: A Cart shall display relevant course information, including course name, course number, units, instructor name, meeting time, location, and enrollment status for each section.
- FR 21.3: A Cart shall allow a Student to add or remove Class Sections.
- FR 21.4: A Cart shall calculate the total number of course units.

Degree Progress Tree

- FR 2.12: A Student shall be able to view a Degree Progress Tree based on their declared Major.
- FR 2.13: A Student shall be able to update their declared Major for major planning purposes.
- FR 2.14: A Student shall be able to view completed, planned, and remaining degree requirements.
- FR 2.15: A Student shall be able to view missing prerequisites for planned courses.
- FR 7.5: A Planned Schedule shall identify missing prerequisites for selected Class Sections.
- FR 7.6: A Planned Schedule shall support basic walking time checking between selected Class Sections when building information is available.
- FR 7.7: A Planned Schedule shall provide access to the Calendar view.
- FR 9.1: A Degree Progress Tree shall display required General Education courses, major electives, and major requirements.
- FR 9.2: A Degree Progress Tree shall display completed requirements, planned requirements, and remaining requirements.
- FR 9.3: A Degree Progress Tree shall display remaining units needed for graduation.
- FR 9.4: A Degree Progress Tree shall display prerequisite information for planned courses.
- FR 9.5: A Degree Progress Tree shall recommend courses that satisfy remaining requirements.
- FR 9.6: A Degree Progress Tree shall allow a Student to view which Courses can satisfy remaining requirements.
- FR 9.7: A Degree Progress Tree shall update when a Student updates their Planned Schedule.
- FR 9.8: A Degree Progress Tree shall update when a Student changes their declared Major.

- FR 9.9: A Degree Progress Tree shall display which requirement category a planned Course satisfies.

Admin Management

- FR 3.1: An Admin shall be able to add, remove, or update Course information.
 - FR 3.2: An Admin shall be able to add, remove, or update Class Section information.
 - FR 3.3: An Admin shall be able to add, remove, or update Building location information.
 - FR 3.4: An Admin shall be able to add, remove, or update Degree Requirement information.
 - FR 3.6: An Admin shall be able to modify Student profile information.
 - FR 15.1: A Major shall have a major name, related Department, required units, and catalog year.
 - FR 15.2: A Major shall show the Degree Requirements.
 - FR 16.1: A Degree Requirement shall have a requirement category, required units, and requirement status.
 - FR 16.2: A Degree Requirement shall be satisfied by Courses.
 - FR 16.3: A Degree Requirement shall indicate whether the requirement is completed, planned, or remaining.
-

Priority 3

Enrollment Information and Enrollment Records

- FR 2.7: A Student shall be able to view enrollment information for the current term.
- FR 2.9: A Student shall be able to enroll in, add, drop, or withdraw from Courses when enrollment is open for their student group.
- FR 2.22: A Student shall be able to view Enrollment Records from different semesters.
- FR 4.1: Enrollment Information shall display the date and time when the Student's student type is eligible to begin enrollment.
- FR 4.2: Enrollment Information shall display the enrollment window for the current term.
- FR 4.3: Enrollment Information shall notify a Student when enrollment is available for their student type.
- FR 4.4: Enrollment Information shall display the add/drop deadline for the current term.
- FR 4.5: Enrollment Information shall display the withdrawal deadline for the current term.
- FR 10.1: An Enrollment Record shall display a Student's academic history across semesters.
- FR 10.2: An Enrollment Record shall display the Semester associated with each Course.
- FR 10.3: An Enrollment Record shall display the grade associated with each Course.

Notifications

- FR 2.23: A Student shall be able to view and dismiss Notifications.
- FR 19.1: A Notification shall inform a Student when their enrollment period begins.
- FR 19.2: A Notification shall inform a Student about approaching deadlines.
- FR 19.3: A Notification shall warn a Student about schedule conflicts or insufficient walking time between classes.
- FR 19.4: A Notification shall inform a Student when an attempted action succeeds or fails.

Financial Data

- FR 2.17: A Student shall be able to view outstanding charges.
- FR 2.18: A Student shall be redirected to the appropriate payment portal to make payments for outstanding charges.
- FR 2.19: A Student shall be redirected to the financial aid website for application.
- FR 2.20: A Student shall be able to view their financial aid awards.
- FR 2.21: A Student shall be able to accept or decline their financial aid awards.
- FR 3.5: An Admin shall be able to add, remove, or update student financial information.
- FR 11.1: Student Charges shall display charge information, including charge type, amount, and due date.
- FR 11.2: Student Charges shall display the total outstanding balance.
- FR 11.3: Student Charges shall display the estimated amount owed or refund amount after financial aid is applied.
- FR 11.4: Student Charges shall provide a link to the university payment portal for payment processing.
- FR 12.1: Financial Aid shall display name, type, term, amount, and status.
- FR 12.2: Financial Aid shall be applied toward outstanding charges when applicable.
- FR 12.3: Financial Aid shall provide a link or redirect to the university financial aid application website.
- FR 17.1: A Billing Profile shall be provided to Students.
- FR 17.2: A Billing Profile shall store billing contact information and payment-related account information.
- FR 17.3: A Billing Profile shall display all outstanding charges, payments, refunds, and financial aid credits.
- FR 17.4: A Billing Profile shall calculate the current outstanding balance.

Maybe Later

- FR 2.10: A Student shall be directed to the appropriate Canvas course page when selecting the Canvas button for an enrolled Course.
- FR 8.5: A Campus Map shall display the department building associated with a Student's declared Major.
- FR 18.1: A Student Profile shall display the User's student information, including name, university email address, role, academic level, student type, and declared Major.

- FR 18.2: A Student Profile shall allow a User to add or update contact information.
- FR 18.3: A Student Profile shall allow a Student to update their declared Major for degree planning purposes.

Out of Scope for Current Version

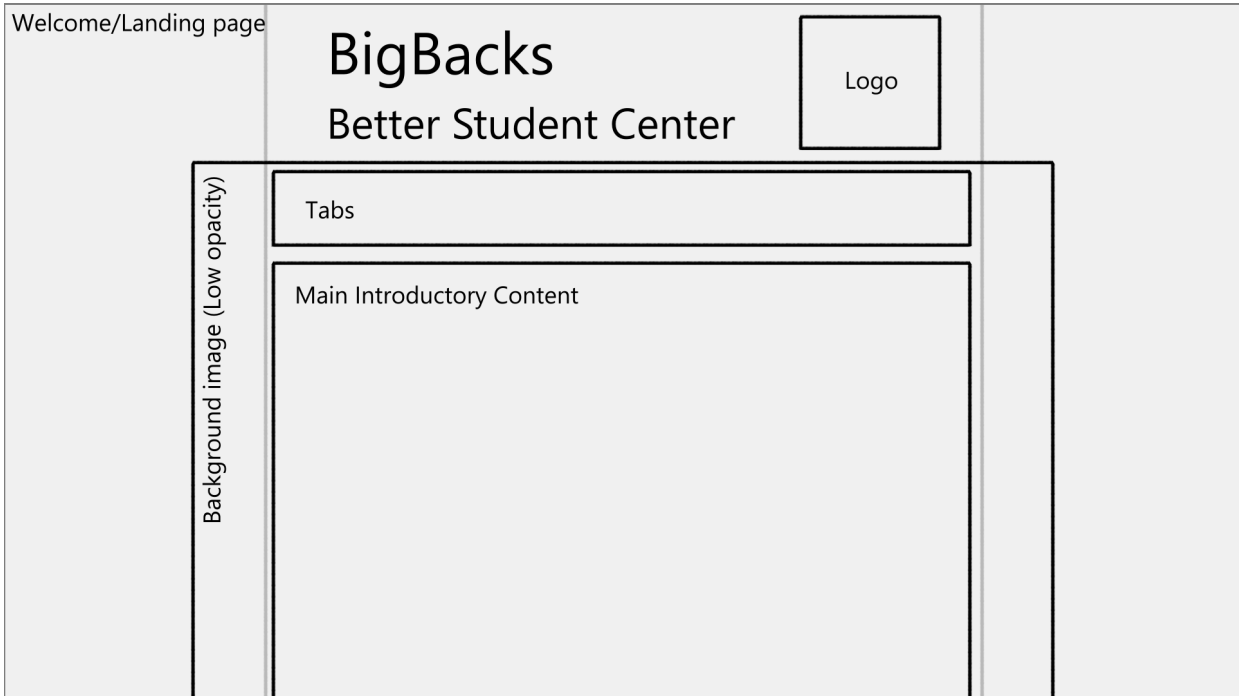
Professor or Instructor accounts are not included as system user roles in the current project scope. Instructor names may still appear as display-only information in Class Section, Calendar, and Course Details views.

Mockups and Storyboards

Storyboard 1: Administrator registration

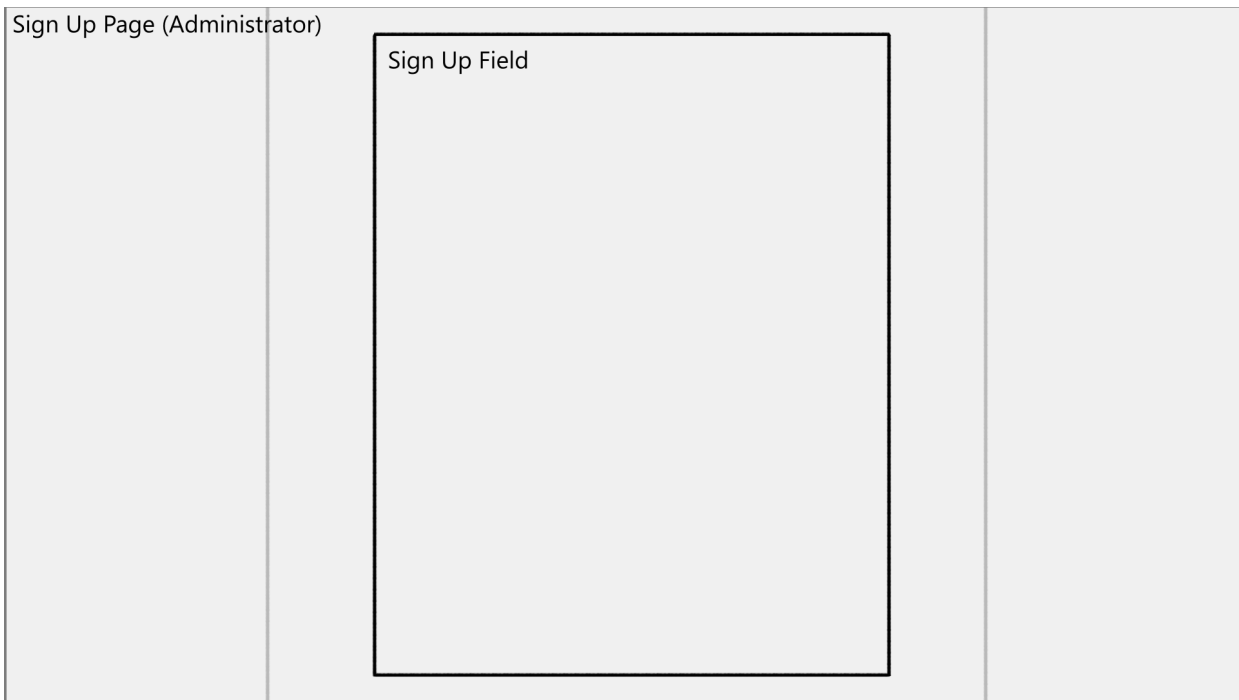
Assumption: An organization wants to register for our service

The organization expects a straightforward process, and wants to read about our services

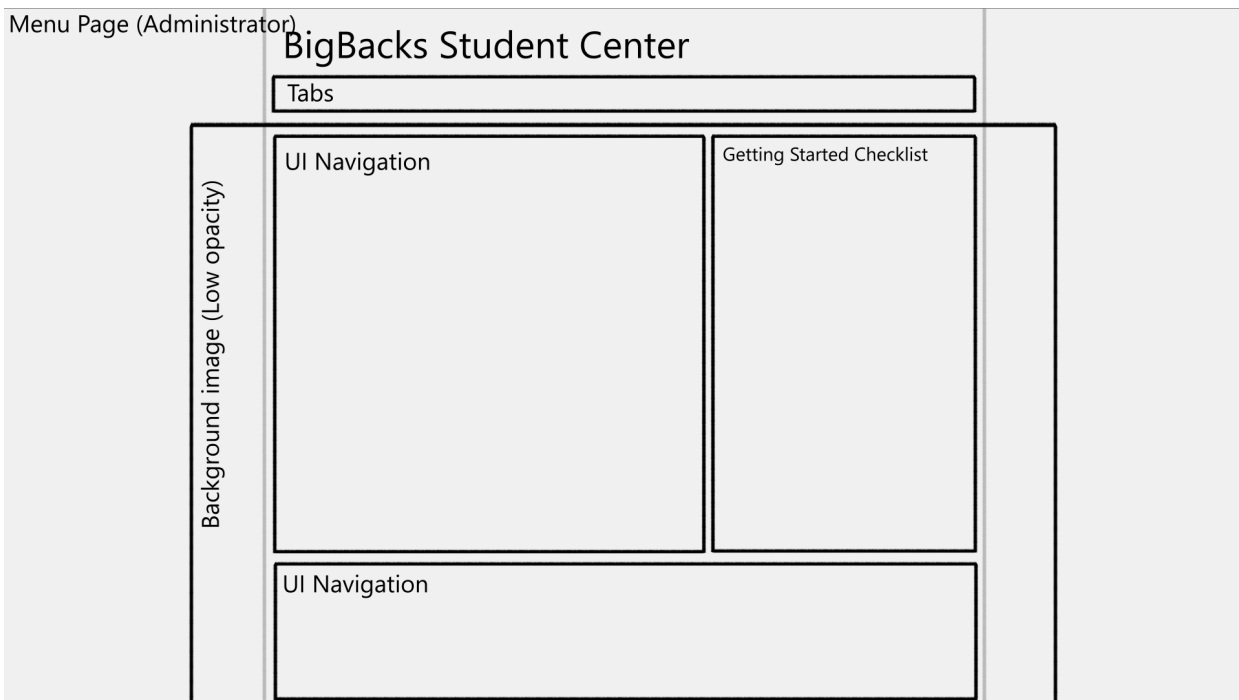


A user wants to begin enrolling a school in our application

Visiting the site will show them an overview and a “Sign Up” button in the tabs, as well as a “sign up” button somewhere in the next that acts as an easy way to be incentivized to use our services



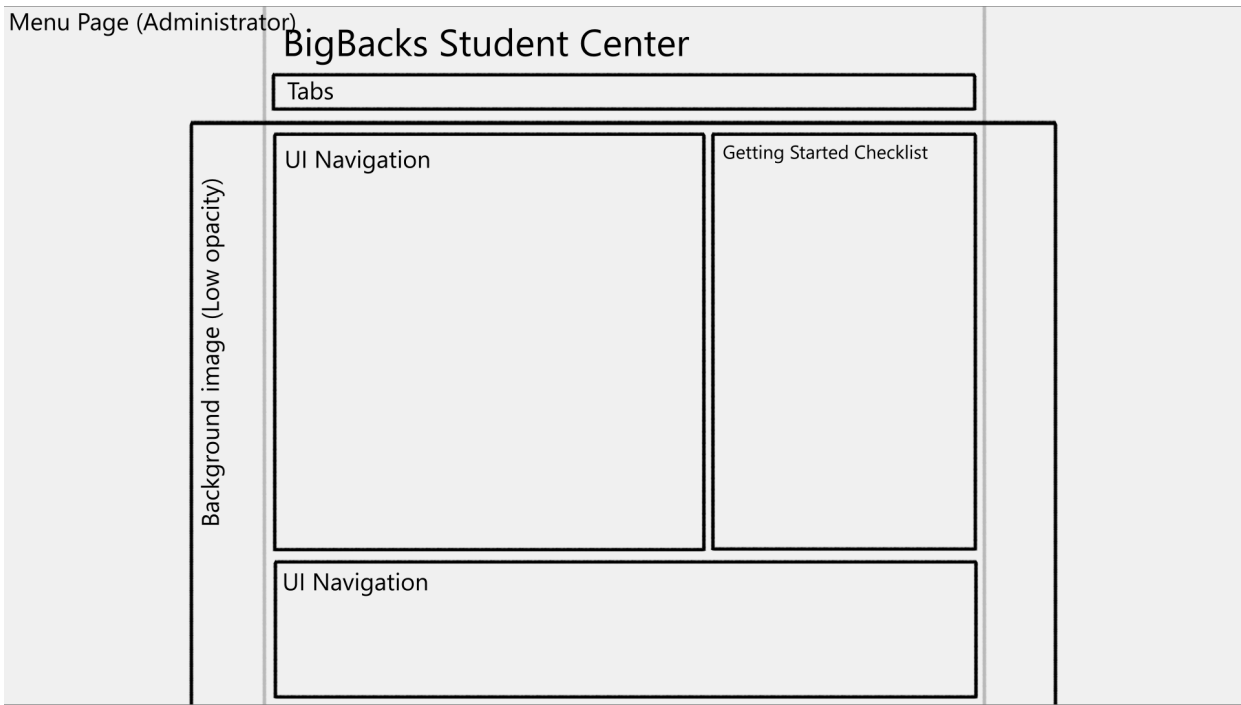
Visiting the Sign Up / Login Page will prompt them for information to create an account (MIGHT be changed later in favor of direct email registration for org admins)



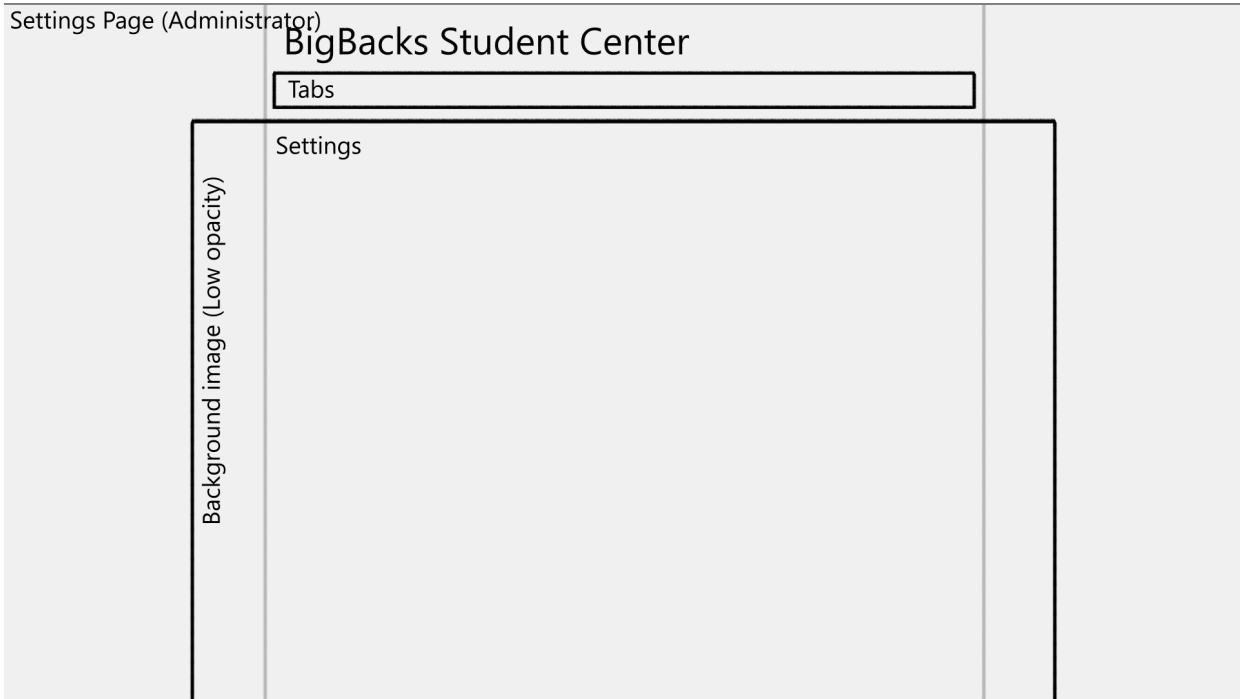
After creating an account, the menu page will be presented to them

Storyboard #2: Administrator wants to customize preferences

Assumption: The admin wants to change a few settings to fit their preferences or view their data. The admin is also logged into the site.



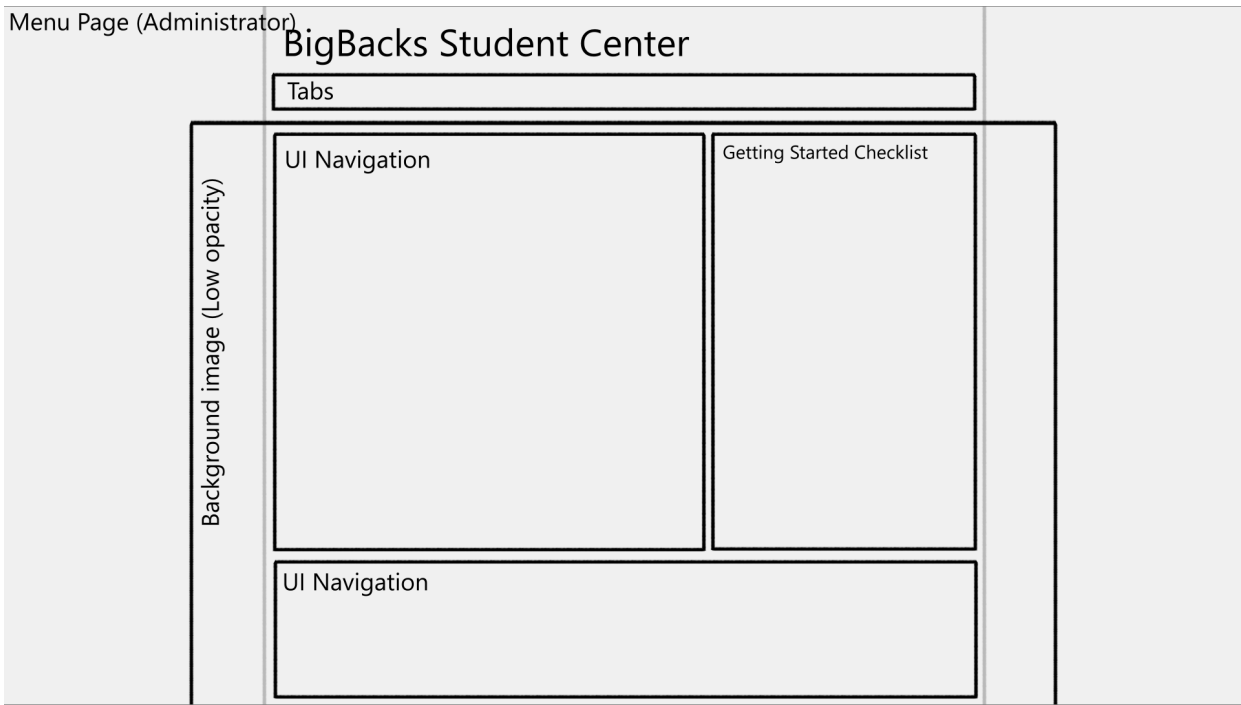
From the Main menu, they can be directed to the settings via a small dropdown that is accessible in the tabs menu. This will open up the settings page for the administrator.



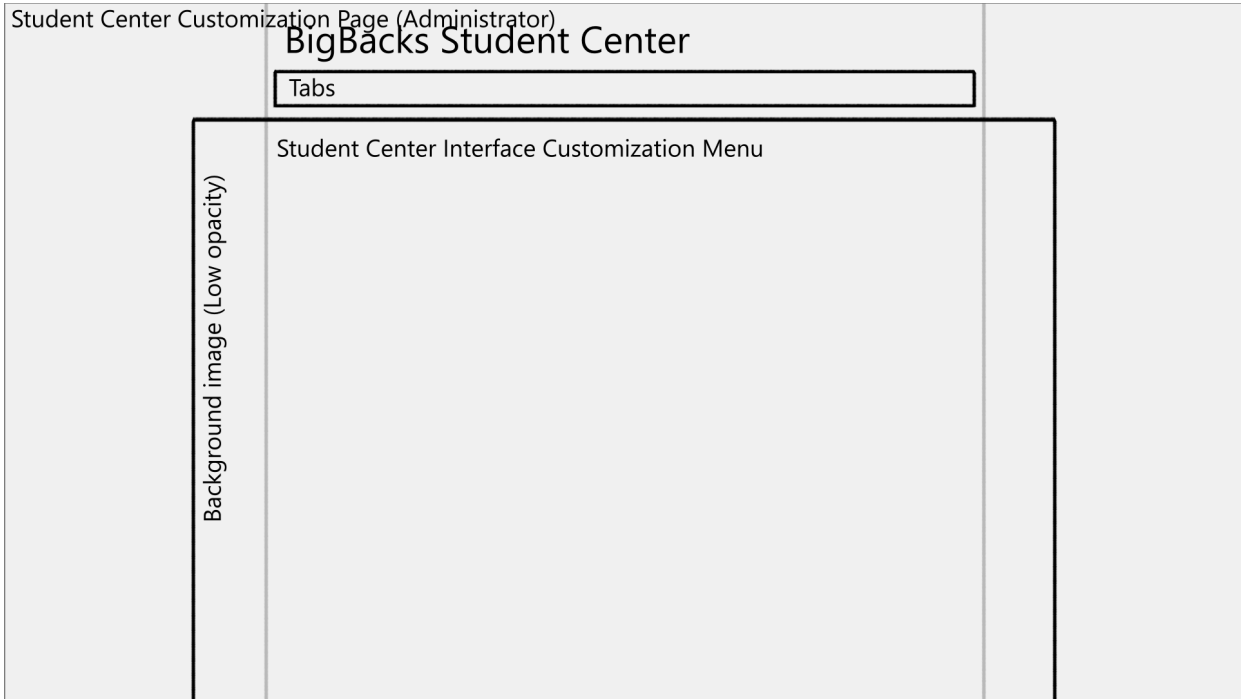
Here will be where they can access the settings.

Storyboard #3: Admin wants to customize the interface for the student

Assumption: The admin wants to change the appearance of the site for the student.
The admin is also logged into the site.



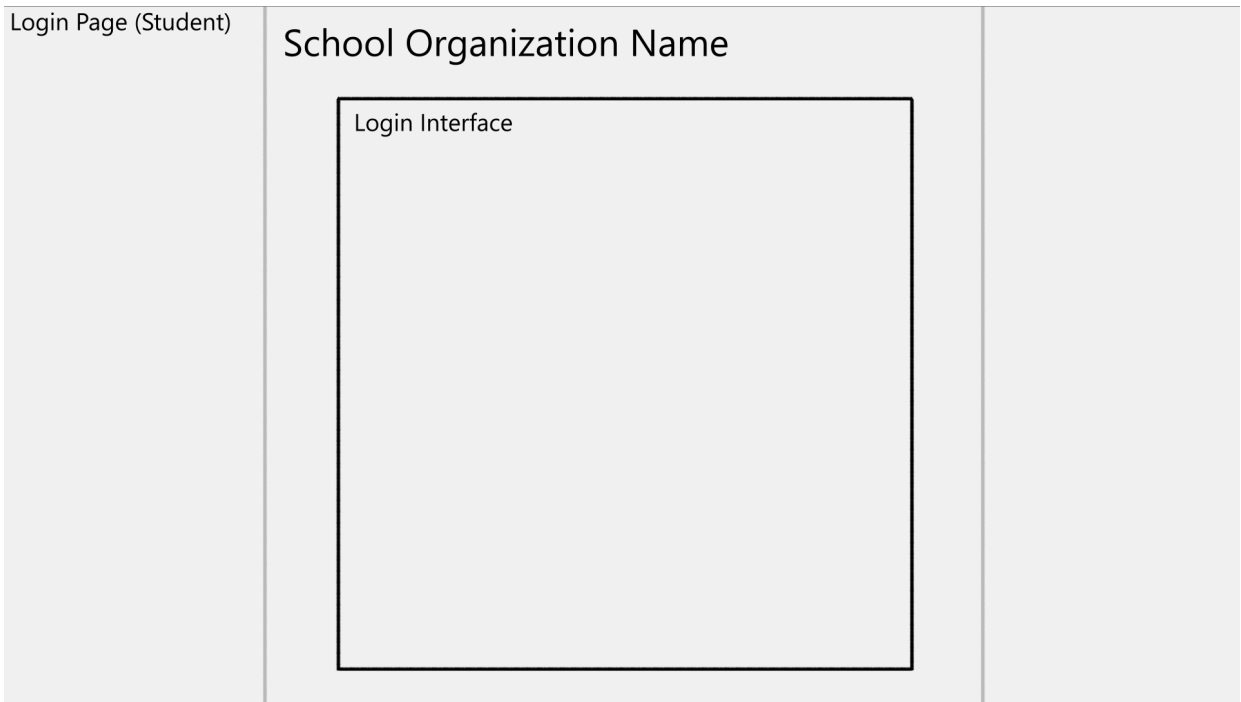
From the main menu, the admin can access the button that takes them to the student interface customization



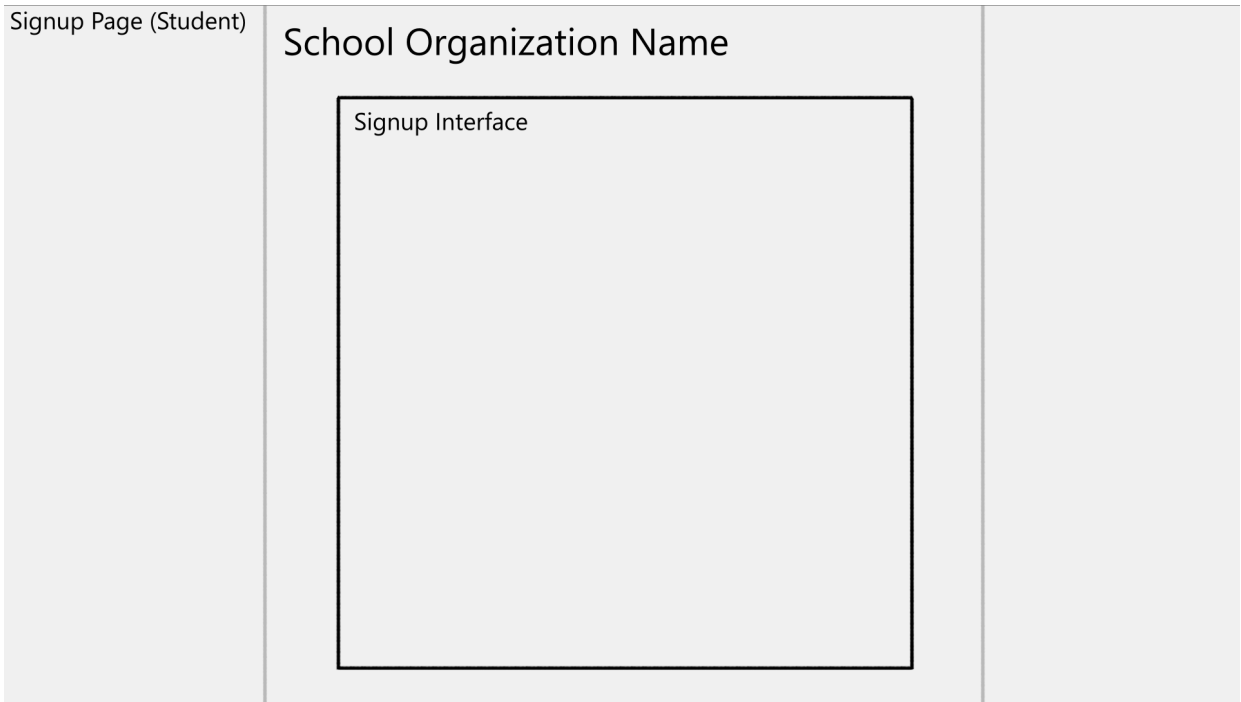
Storyboard #4: Student wants to log in with their student ID

assumptions: student knows their affiliated Student ID as well as a randomly generated password privately sent to them by the school organization

OR: Student has already reset their password and only wants to log in



When a student wants to sign in to the school’s student center, they will access the site’s login link and be asked to input their Student ID and password



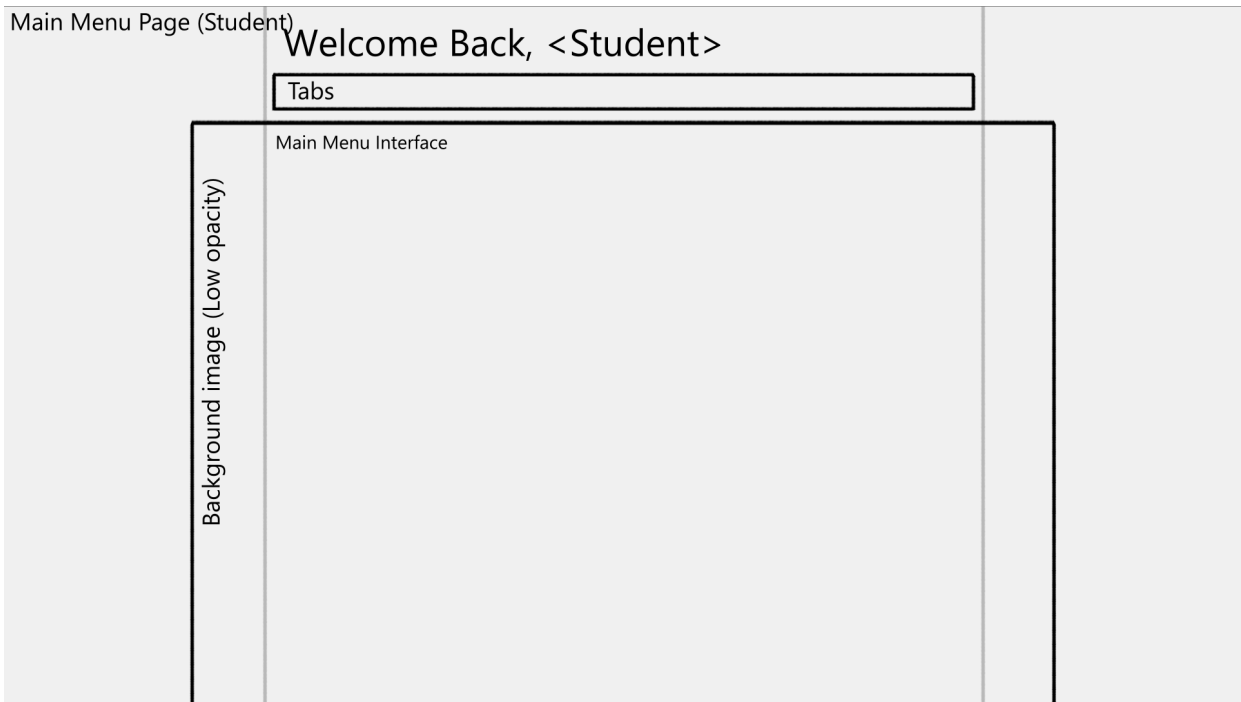
(We might consider removing the sign up page for students)



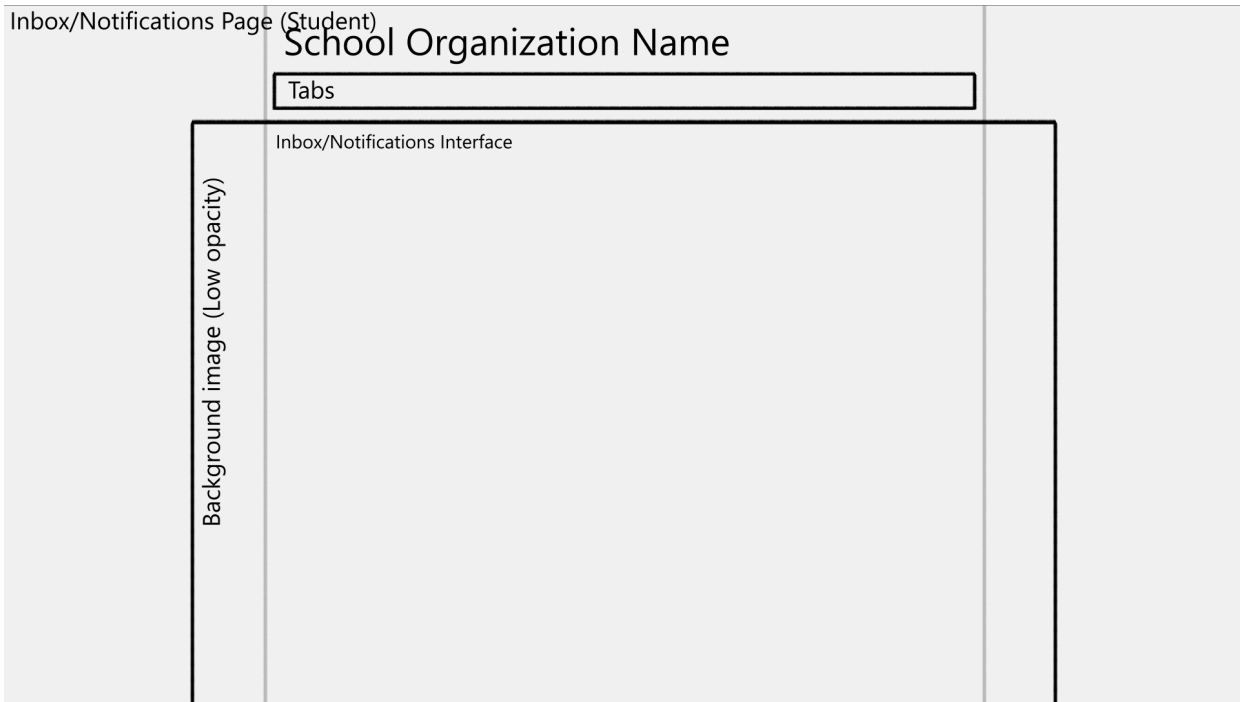
Afterwards, Student will be presented with the main menu, which includes all the hud elements that may or may not have been customized already by the admin or student.

Storyboard #5: Student wants to view notifications, settings, or course information on their account, or ask for help

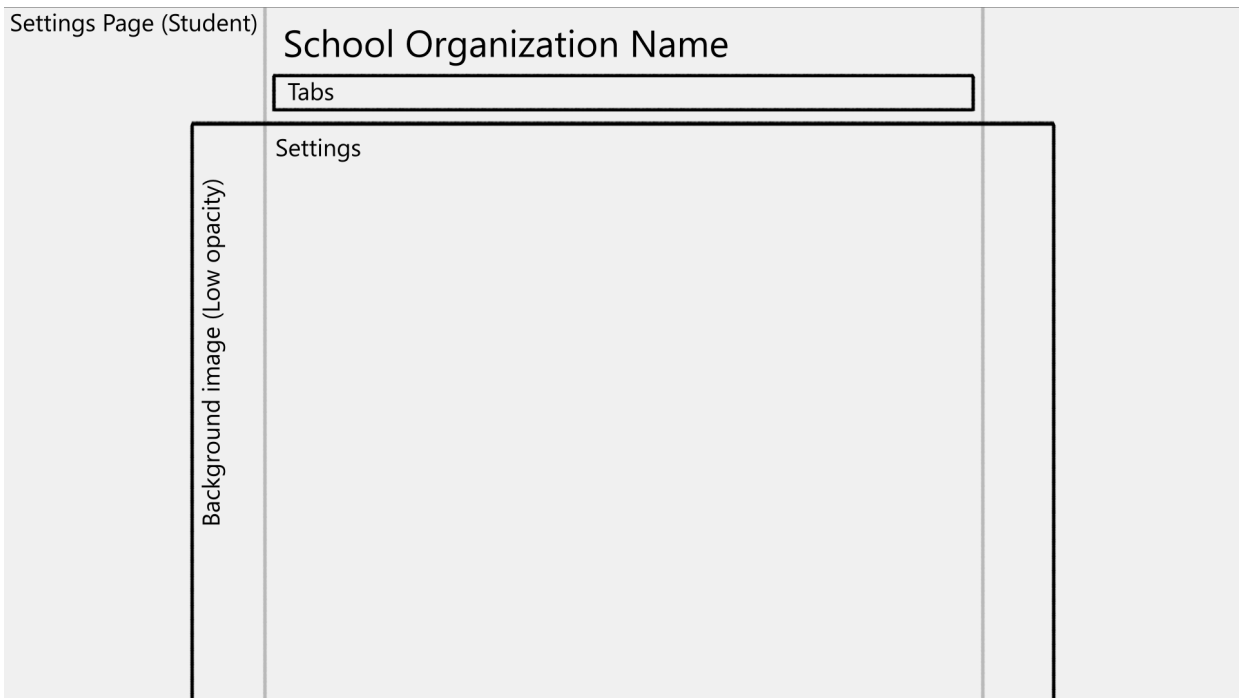
Assumptions: Student is signed in and on the main student page



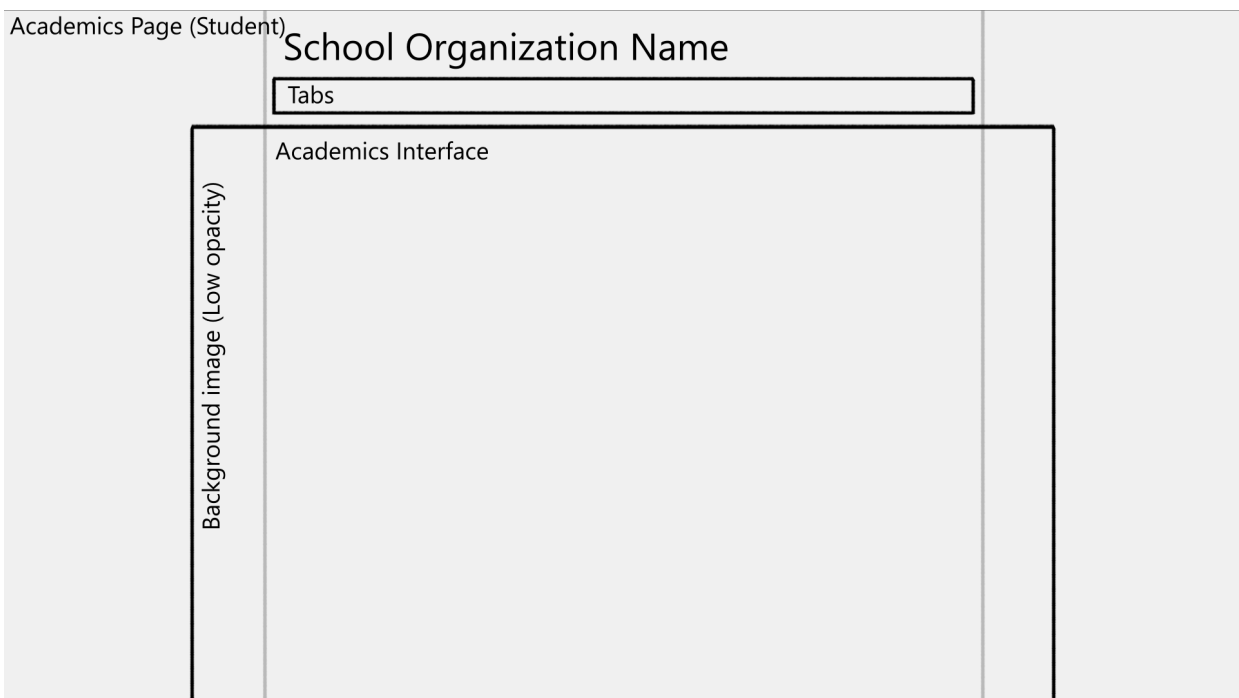
Student is on the main menu page and wants to view their notifications. Notifications will be located on the regular hud elements with a small indicator that shows how many new unread notifications exist.



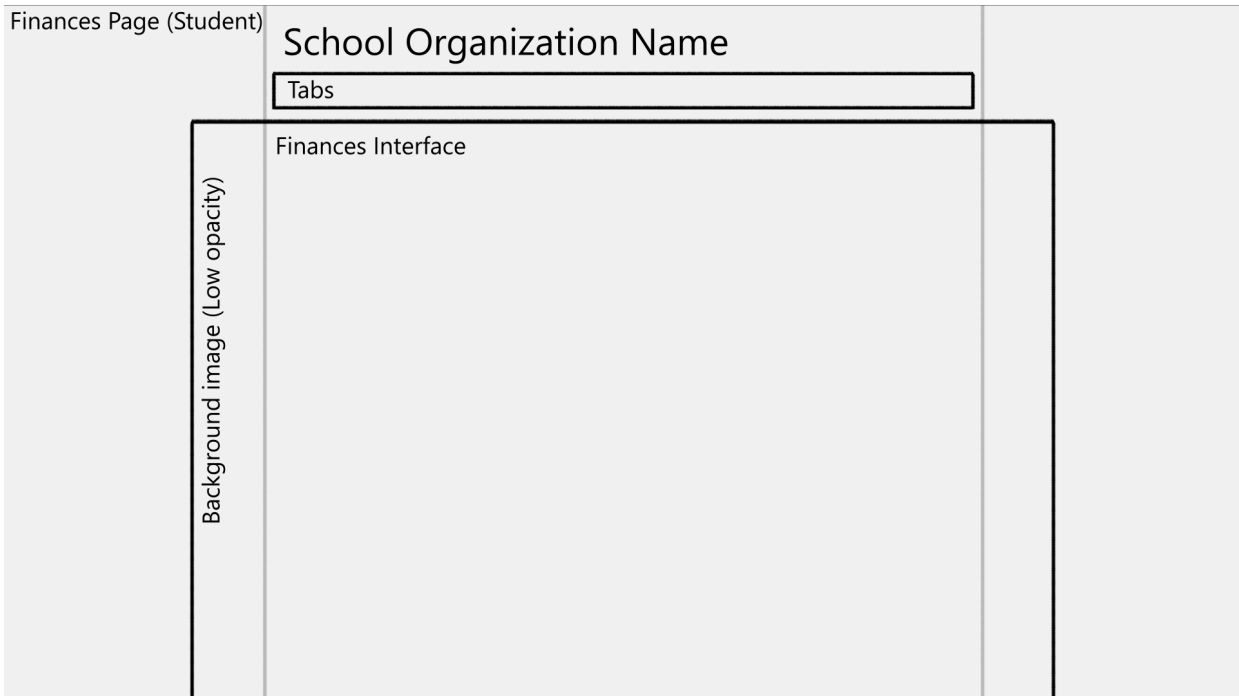
After clicking notifications, they can see the interface for notifications



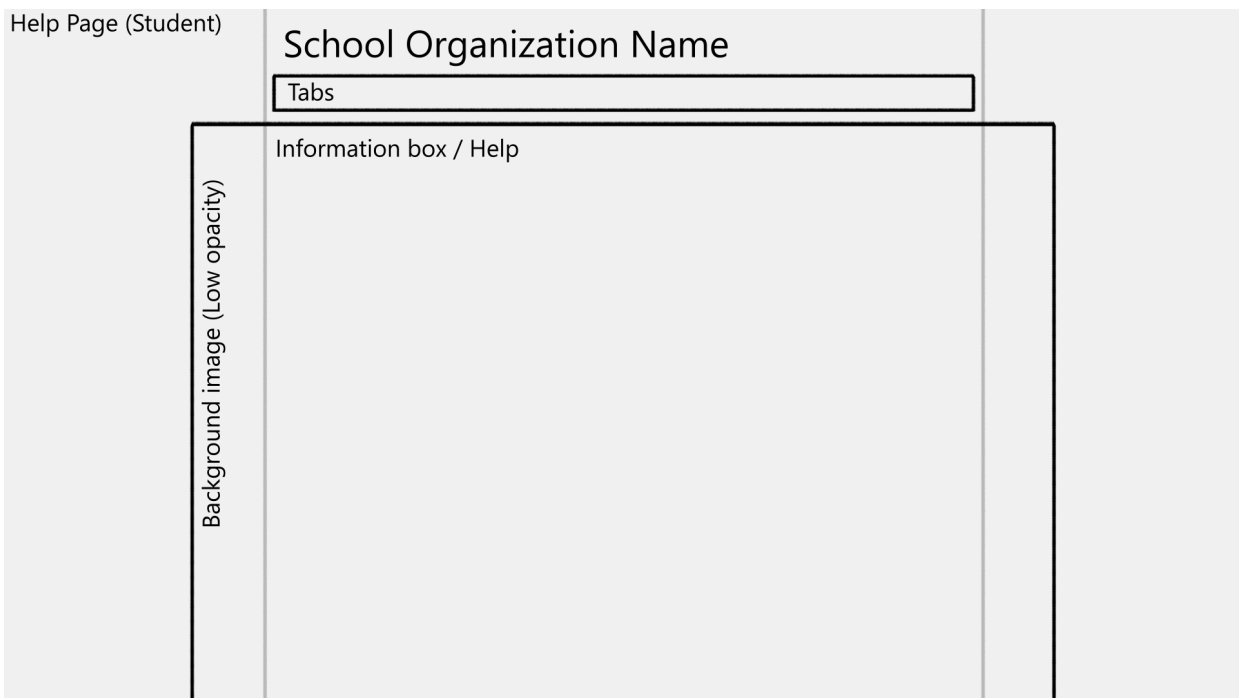
Similarly for settings, which will be available via the settings button on a dropdown



Similar to notifications, Academics is available as a hud button, which shows everything academic related

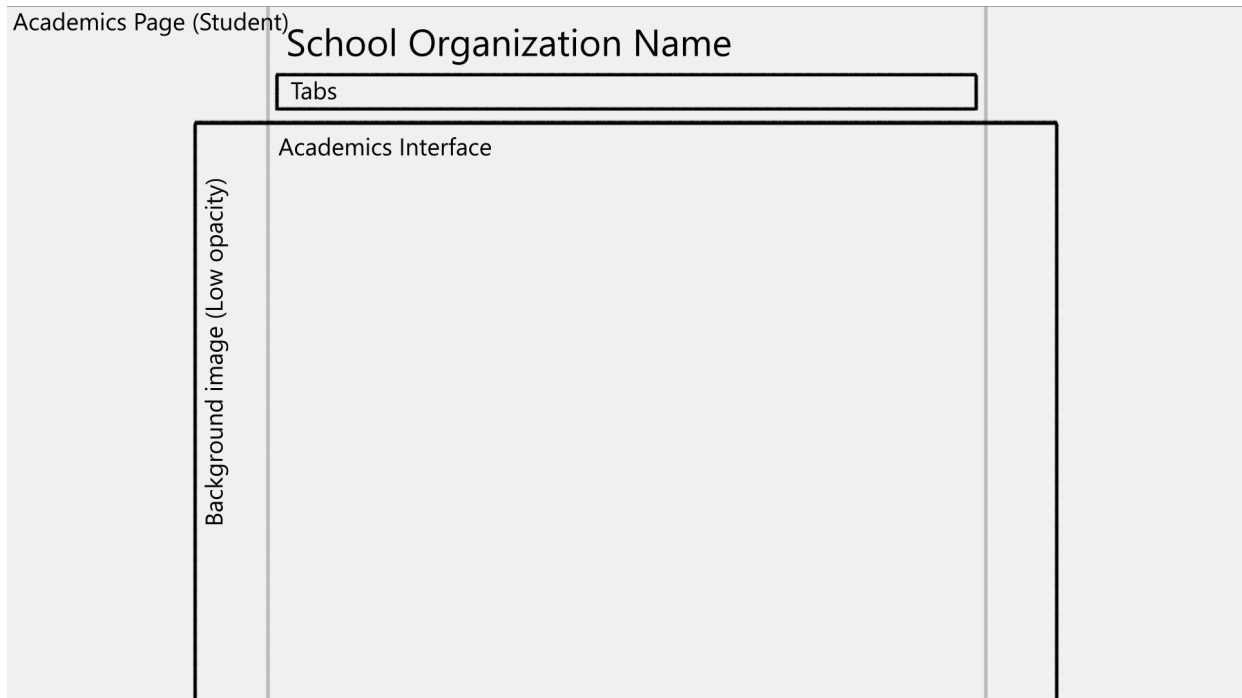


And similarly for finances

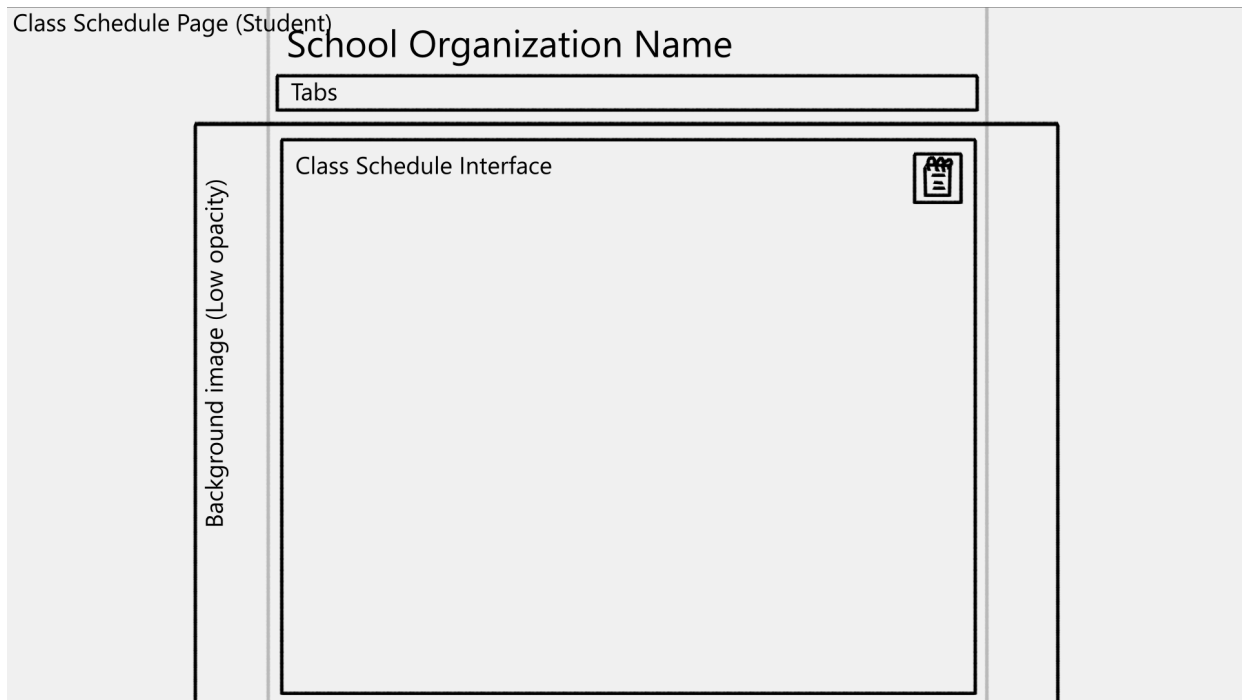


This page will depend on what the school administration decides to include, but will typically include resources for students with questions.

Storyboard #7: Student academics management
Assumptions: Student wants to view, add, drop, or manage courses and manage education progress



The academics interface by default should have all the relevant academic hud elements that are easily accessible by the student. These include campus map, class schedule, enrollment.



School Organization Name

Tabs

Background image (Low opacity)

Class Enrollment Interface

School Organization Name

Tabs

Background image (Low opacity)

Campus Map Interface

High Level System Design

Database Architecture

Functional Requirements

Entity Name/table_name(Type)

1. User/app_user(Strong)

- a. A User shall have exactly one unique user ID.
- b. A User shall reference exactly one school ID.
- c. A User shall have exactly one institutional email address.
- d. A User shall have exactly one password hash.
- e. A User shall have exactly one first name.
- f. A User shall have exactly one last name.
- g. A User shall have exactly one role (student or admin)
- h. A User shall have exactly one created timestamp.
- i. A User may have zero or one last login timestamp.
- j. A User may receive zero or many Notifications.

2. Student/student (Strong)

- a. A Student shall reference exactly one user ID.
- b. A Student shall have exactly one academic level (freshman, sophomore, junior, senior, graduate).
- c. A Student shall have exactly one student type (first-time, continuing, transfer, international)
- d. A Student shall reference exactly one DegreeProgram.
- e. A student shall have exactly one total credits.
- f. A Student shall have exactly one expected graduation term.
- g. A Student shall have exactly one city.
- h. A Student shall have exactly one street.
- i. A Student shall have exactly one state.
- j. A Student shall have exactly one zipcode.
- k. A Student may have zero or one phone number.
- l. A Student shall have exactly one walking-speed value. (in meters/second)
- m. A Student may have zero or many Enrollments.
- n. A Student may have zero or many PlannedCourses.
- o. A Student may have zero or many items in ClassCart.
- p. A Student may have zero or one BillingProfile.
- q. A Student may have zero or many StudentCharges.
- r. A Student may have zero or many FinancialAid awards.
- s. A Student may have zero or many StudentHolds.
- t. A Student may be associated with zero or many classes through PlannedCourse.

3. Admin/admin (Strong)

- a. An Admin shall reference exactly one user ID.

- b. An Admin shall have exactly one unique institutional employee ID.
- 4. **InterfaceSetting/interface_setting (Strong)**
 - a. An InterfaceSetting shall have exactly one unique setting ID.
 - b. An InterfaceSetting shall reference exactly one admin ID.
 - c. An InterfaceSetting shall have exactly one setting name. (header, sidebar...)
 - d. An InterfaceSetting shall have exactly one setting value. (color, number, text)
 - e. An InterfaceSetting shall have exactly one last updated timestamp.
- 5. **School/school (Strong)**
 - a. A School shall have exactly one unique school ID.
 - b. A School shall have exactly one name.
 - c. A School may have zero or many Users.
 - d. A School may have zero or many Departments.
 - e. A School may have zero or many Buildings.
 - f. A School may have zero or many Semesters.
 - g. A School may have zero or many GEAreas.
- 6. **Department/department (Strong)**
 - a. A Department shall have exactly one unique department ID.
 - b. A Department shall reference exactly one school ID.
 - c. A Department shall have exactly one department name.
 - d. A Department shall reference exactly one building ID.
 - e. A Department shall have exactly one office email address.
 - f. A Department shall have exactly one office phone number.
 - g. A Department may have zero or many Majors.
 - h. A Department may have zero or many Courses.
 - i. A Department (school ID, department name) pair shall be unique.
- 7. **Major/major (Strong)**
 - a. A Major shall have exactly one unique major ID.
 - b. A Major shall reference exactly one Department.
 - c. A Major shall have exactly one name.
 - d. A Major may have zero or many DegreePrograms.
- 8. **DegreeProgram/degree_program (Strong)**
 - a. A DegreeProgram shall have exactly one unique degree program ID.
 - b. A DegreeProgram shall reference exactly one major ID.
 - c. A DegreeProgram shall have exactly one degree type (BA, BS, BFA, MA, MS, or PHD)
 - d. A DegreeProgram shall have exactly one catalog year.
 - e. A DegreeProgram shall have one required major units value.
 - f. A DegreeProgram shall have one required general education units value.
 - g. A DegreeProgram (major ID, degree type, catalog year) triple shall be unique.
- 9. **Semester/semester (Strong)**
 - a. A Semester shall have exactly one unique semester ID.
 - b. A Semester shall reference exactly one school ID.
 - c. A Semester shall have exactly one term year.
 - d. A Semester shall have exactly one term type (Fall, Spring, Summer, Winter)
 - e. A Semester shall have exactly one start date.
 - f. A Semester shall have exactly one end date.
 - g. A Semester shall have exactly one add/drop deadline.

- h. A Semester shall have exactly one withdrawal deadline.
- i. A Semester (school ID, term year, term type) combination shall be unique.

10. **Course/course (Strong)**

- a. A Course shall have exactly one unique course ID.
- b. A Course shall reference exactly one department ID.
- c. A Course shall have exactly one subject code.
- d. A Course shall have exactly one course number.
- e. A Course shall have exactly one course title.
- f. A Course shall have exactly one course description.
- g. A Course shall have exactly one course units value.
- h. A Course shall have exactly one course level (lower division, upper division, or graduate)
- i. A Course shall have exactly one repeatable status.
- j. A Course shall have exactly one section type (lecture, lab).
- k. A Course may be associated with zero or more DegreeRequirement through RequiredCourse.
- l. A Course may be associated with zero or more courses through CoursePrerequisite.
- m. A Course may be associated with zero or many GEAreas through CourseGEArea.

11. **ClassSection/class_section (Strong)**

- a. A ClassSection shall have exactly one unique class section ID.
- b. A ClassSection shall reference exactly one course ID.
- c. A ClassSection shall reference exactly one semester ID.
- d. A ClassSection shall reference zero or one classroom ID.
- e. A ClassSection shall reference exactly one instructor.
- f. A ClassSection shall have exactly one section number.
- g. A ClassSection shall have exactly one start date.
- h. A ClassSection shall have exactly one end date.
- i. A ClassSection shall have exactly one meeting start time.
- j. A ClassSection shall have exactly one meeting end time.
- k. A ClassSection may have zero or more MeetingDays.
- l. A ClassSection shall have exactly one modality (in-person, online, hybrid)
- m. A ClassSection shall have exactly one meeting type (synchronous/asynchronous)
- n. A ClassSection shall have exactly one capacity.
- o. A ClassSection shall have exactly one enrolled count.
- p. A ClassSection shall have exactly one waitlist capacity.
- q. A ClassSection shall have exactly one waitlist count.
- r. A ClassSection shall have exactly one status (open, closed, waitlist, cancelled)

12. **MeetingDay/meeting_day (Weak)**

- a. A MeetingDay shall reference exactly one ClassSection ID.
- b. A MeetingDay shall have exactly one day of the week.
- c. A MeetingDay (Class section ID, day of week) pair shall be unique.

13. **Instructor/instructor (Strong)**

- a. An Instructor shall have exactly one unique instructor ID.
- b. An Instructor shall have exactly one first name.

- c. An Instructor shall have exactly one last name.
 - d. An Instructor shall have exactly one institutional email.
- 14. Building/building (Strong)**
- a. A Building shall have exactly one unique building ID.
 - b. A Building shall reference exactly one school ID.
 - c. A Building shall have exactly one name.
 - d. A Building may have one or many BuildingTypes.
 - e. A Building shall have zero or more Classrooms.
 - f. A Building shall have exactly one unique map element ID. (for frontend)
 - g. A Building may be associated with one or more buildings in BuildingDistance.
 - h. A Building (school ID, building name) pair shall be unique.
 - i. A Building (school ID, map element ID) pair shall be unique.
- 15. BuildingType/building_type (Weak)**
- a. A BuildingType shall reference exactly one building ID.
 - b. A BuildingType shall have exactly one building type. (academic, gym, library, student center, dining, administration, parking)
 - c. A BuildingType (building ID, building type) pair shall be unique.
- 16. BuildingDistance/building_distance (Associative)**
- a. A BuildingDistance shall reference exactly one origin building ID.
 - b. A BuildingDistance shall reference exactly one destination building ID.
 - c. A BuildingDistance shall have exactly one distance value. (in meters)
 - d. A BuildingDistance (origin building ID, destination building ID) pair shall be unique.
- 17. Classroom/classroom (Strong)**
- a. A Classroom shall have exactly one unique classroom ID.
 - b. A Classroom shall reference exactly one building ID.
 - c. A Classroom shall have exactly one room number.
- 18. EnrollmentWindow/enrollment_window (Strong)**
- a. An EnrollmentWindow shall have exactly one unique window ID.
 - b. An EnrollmentWindow shall reference exactly one semester ID.
 - c. An EnrollmentWindow shall have exactly one student type.
 - d. An EnrollmentWindow shall have exactly one academic level.
 - e. An EnrollmentWindow shall have exactly one enrollment start timestamp.
 - f. An EnrollmentWindow shall have exactly one enrollment end timestamp.
 - g. An EnrollmentWindow (semester_id, student_type, academic_level) triple shall be unique.
- 19. Enrollment/enrollment (Associative)**
- a. An Enrollment shall reference exactly one student ID.
 - b. An Enrollment shall reference exactly one class section ID.
 - c. An Enrollment shall have exactly one enrollment date.
 - d. An Enrollment shall have exactly one enrollment status (enrolled, dropped, withdrawn, completed)
 - e. An Enrollment may have zero or one grade value (A-F, P/NP)
 - f. An Enrollment (student ID, class section ID) pair shall be unique.
- 20. StudentHold/student_hold (Strong)**
- a. A StudentHold shall have exactly one unique hold ID.
 - b. A StudentHold shall reference exactly one student ID.

- c. A StudentHold shall have exactly one hold type.
 - d. A StudentHold shall have exactly one hold status.
 - e. A StudentHold shall have exactly one placed date.
 - f. A StudentHold may have zero or one resolved date.
 - g. A StudentHold shall have exactly one enrollment-blocking indicator.
- 21. PlannedCourse/planned_course (Associative)**
- a. A PlannedCourse shall reference exactly one student ID.
 - b. A PlannedCourse shall reference exactly one course ID.
 - c. A PlannedCourse shall reference exactly one semester ID.
 - d. A PlannedCourse shall have exactly one added date.
 - e. A PlannedCourse (student ID, course ID, semester ID) combination shall be unique.
- 22. ClassCart/class_cart (Associative)**
- a. A ClassCart shall reference exactly one student ID.
 - b. A ClassCart shall reference exactly one class section ID.
 - c. A ClassCart shall have exactly one added date.
 - d. A ClassCart(student ID, class section ID) pair shall be unique.
- 23. CoursePrerequisite/course_prerequisite (Associative)**
- a. A CoursePrerequisite shall reference exactly one course ID.
 - b. A CoursePrerequisite shall reference exactly one prerequisite course ID.
 - c. A CoursePrerequisite may have zero or one minimum grade required.
 - d. A CoursePrerequisite (course ID, prerequisite course ID) pair shall be unique.
- 24. DegreeRequirement/degree_requirement (Strong)**
- a. A DegreeRequirement shall have exactly one unique degree requirement ID.
 - b. A DegreeRequirement shall reference exactly one degree program ID.
 - c. A DegreeRequirement may reference zero or one GE area ID.
 - d. A DegreeRequirement shall have exactly one requirement name.
 - e. A Degree requirement shall have exactly one requirement type.(major-core, major-elective, GE Area, university-requirement)
 - f. A DegreeRequirement shall have exactly one required units value.
 - g. A DegreeRequirement may have zero or one minimum grade required.
- 25. CourseGEArea/course_gearea (Associative)**
- a. A CourseGEArea shall reference exactly one course ID.
 - b. A CourseGEArea shall reference exactly one GE area ID.
 - c. A CourseGEArea (course ID, GE area ID) pair shall be unique.
- 26. RequiredCourse/required_course (Associative)**
- a. A RequiredCourse shall reference exactly one requirement ID.
 - b. A RequiredCourse shall reference exactly one course ID.
 - c. A RequiredCourse (requirement_id, course_id) pair shall be unique.
- 27. StudentCharge/student_charge (Strong)**
- a. A StudentCharge shall have exactly one unique charge ID.
 - b. A StudentCharge shall reference exactly one student ID.
 - c. A StudentCharge shall reference exactly one semester ID.
 - d. A StudentCharge shall have exactly one charge type (tuition, insurance, parking, student pass, ...etc)
 - e. A StudentCharge shall have exactly one charge amount. (in USD)
 - f. A StudentCharge shall have exactly one due date.

- g. A StudentCharge shall have exactly one charge type (tuition, fees, housing, meal plan, parking, insurance, student pass, or other).
- h. A StudentCharge shall have exactly one charge status (pending, paid, overdue, waived, refunded, or cancelled).

28. FinancialAid/financial_aid (Strong)

- a. A FinancialAid shall have exactly one unique aid ID.
- b. A FinancialAid shall reference exactly one student ID.
- c. A FinancialAid shall reference exactly one semester ID.
- d. A FinancialAid shall have exactly one aid type (grant, loan, scholarship, work-study).
- e. A FinancialAid shall have exactly one aid name.
- f. A FinancialAid shall have exactly one amount. (in USD)
- g. A FinancialAid shall have exactly one status (pending, accepted, declined, disbursed).
- h. A FinancialAid may have zero or one disbursement date.
- i. A FinancialAid may have zero or one accepted date.

29. BillingProfile/billing_profile (Strong)

- a. A BillingProfile shall have exactly one unique billing profile ID.
- b. A BillingProfile shall reference exactly unique one student ID.
- c. A BillingProfile shall have exactly one city.
- d. A BillingProfile shall have exactly one street.
- e. A BillingProfile shall have exactly one state.
- f. A BillingProfile shall have exactly one zipcode.
- g. A BillingProfile may have zero or more BillingTransactions.

30. BillingTransaction/billing_transaction (Strong)

- a. A BillingTransaction shall have exactly one unique transaction ID.
- b. A BillingTransaction shall reference exactly one billing profile ID.
- c. A BillingTransaction shall have exactly one transaction type (payment, refund, or financial aid)
- d. A Billing Transaction shall have exactly one transaction amount (in USD)
- e. A BillingTransaction shall have exactly one transaction timestamp.
- f. A Billing Transaction shall have exactly one status (pending , completed, or failed)

31. Notification/notification (Strong)

- a. A Notification shall have exactly one unique notification ID.
- b. A Notification shall reference exactly one user ID.
- c. A Notification shall exactly one notification type (enrollment, deadline, schedule-conflict, walking-time-conflict, payment, general)
- d. A Notification shall have exactly one title.
- e. A Notification shall have exactly one message.
- f. A Notification shall have exactly one created timestamp.

Entity Descriptions

Type	Entities	Description
User and Access Management	School, User, Student, Admin, InterfaceSetting	Stores schools, user accounts, student and administrator information, and interface settings managed by administrators.
Academic Organization	Department, Major, DegreeProgram, GEArea	Stores the academic structure of each school, including departments, majors, degree programs, catalog years, and general education areas.
Course Catalog	Course, CoursePrerequisite, CourseGEArea	Stores course information, prerequisite relationships, and the general education areas associated with each course.
Degree Requirements	DegreeRequirement, RequiredCourse	Stores graduation requirement categories and the courses that may satisfy each requirement.
Class Scheduling	Semester, ClassSection, MeetingDay, Instructor	Stores academic terms and specific course offerings, including instructors, meeting dates, times, days, modality, capacity, and enrollment availability.
Enrollment and Course Planning	EnrollmentWindow, Enrollment, StudentHold, PlannedCourse, ClassCart	Stores enrollment periods, student enrollment records, enrollment-blocking holds, long-term course plans, and class sections selected for registration.
Campus Navigation	Building, BuildingType, Classroom, BuildingDistance	Stores campus buildings, building categories, classrooms, and distances between buildings.
Student Financial Information	StudentCharge, FinancialAid, BillingProfile, BillingTransaction	Stores student charges, financial aid awards, billing addresses, and payment, refund, or financial-aid transaction records.

Notifications	Notification	Stores system-generated messages about enrollment, deadlines, schedule conflicts, walking-time conflicts, payments, and other important updates.
---------------	--------------	--

Several key features of our application – Calendar, Degree Planner, and Campus Map – are not stored as database entities. They are user-interface features generated from the entities above.

The Calendar uses class-section, meeting-day, enrollment, and cart data; the Degree Planner uses degree-program, degree-requirement, enrollment, and planned-course data; and the Campus Map uses building, classroom, map-element, and building-distance data.

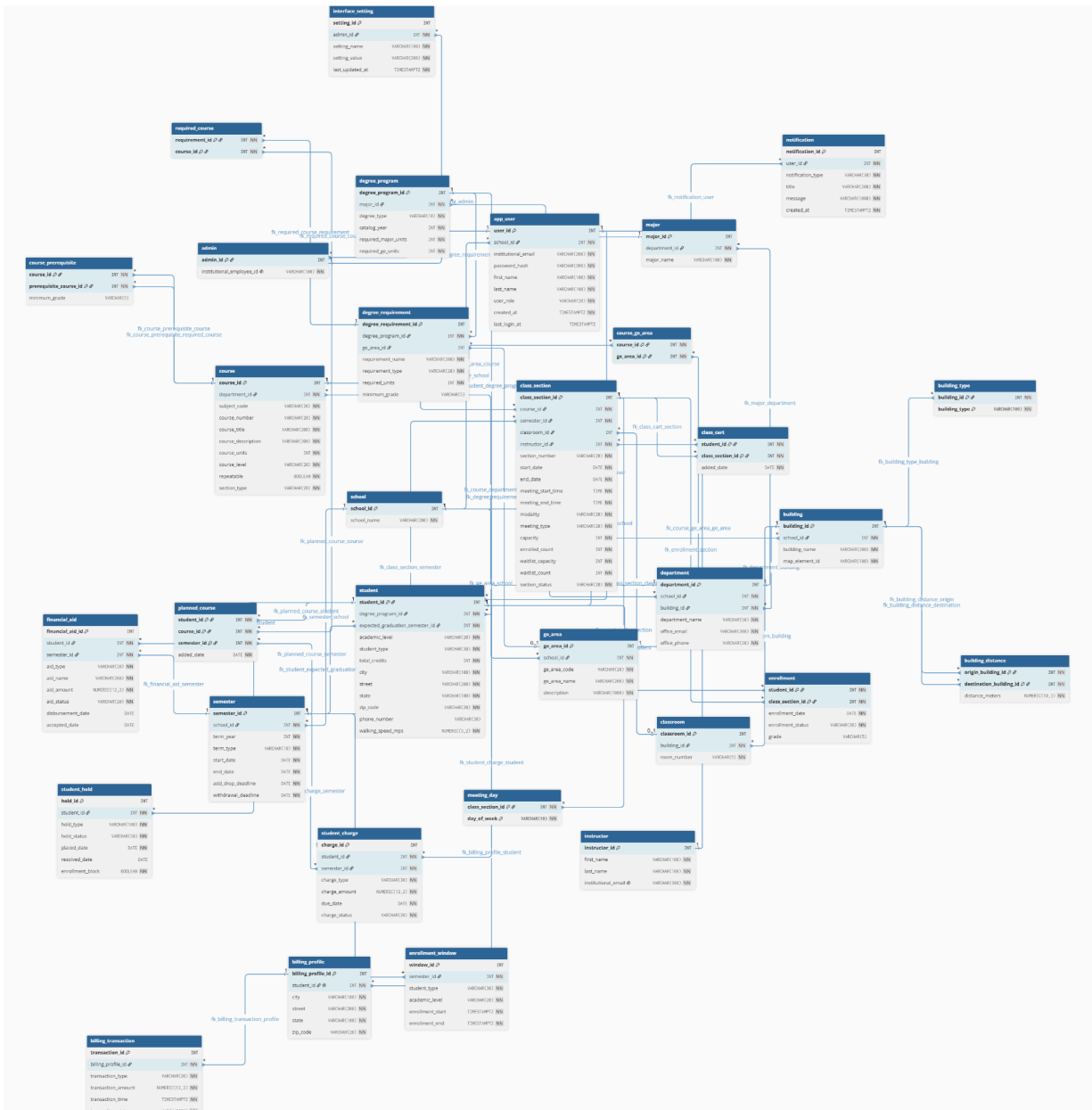
DBMS Selection

The DBMS of our choice is PostgreSQL hosted through Amazon RDS. PostgreSQL is a relational database management system that allows us to store highly structured and related data, which is why we chose it to store students, courses, class sections, enrollments, buildings, and degree requirements data.

PostgreSQL supports primary keys, foreign keys, constraints, transactions, and relational queries, which are necessary for maintaining consistent academic and enrollment data.

Amazon RDS was selected to host PostgreSQL because it provides a managed cloud database that can be accessed by the Node.js and Express backend deployed on AWS EC2.

ERD (Entity-Relationship Diagram)



Media Storage Strategy

As with most student center apps, there will not be many media files for our application. That said, any static media will be stored directly in the frontend's asset directory.

The Degree Progress Tree of our application will be generated dynamically by the frontend using entity data from DegreeProgram, DegreeRequirement, RequiredCourse, Enrollment, and PlannedCourse records.

The Campus Map may be stored as an SVG file. Each Building will have a map element ID that connects the database to the correct building element on the frontend map.

The Calendar will also be generated dynamically by the frontend using data from ClassSection, MeetingDay, Enrollment, and ClassCart records. The database will store the dates, meeting days, meeting times, locations, and enrollment information required to display each class in the correct position on the Calendar.

This is the project's initial media storage strategy; the team will adjust our approach as we go further into the project. The team is fully aware that the implementation of our unique features will not be as straightforward as described, but we do not want to limit ourselves, so we will treat this as a starting point and only refine our strategy when we encounter obstacles during implementation.

Backend Architecture

Reliability

Our student center architecture ensures reliability through allocating different responsibilities and managing cloud services. The application is deployed using an AWS EC2 Ubuntu server that hosts React/Vite frontend through Nginx and Node.js/Express as the backend. Each component of our application can be managed independently, since we separated our application server from the PostgreSQL database ensuring that the data won't be affected as the application is updated.

Our database is hosted on Amazon RDS instead of our EC2 instance. Using a managed database allows us to have automated backups, infrastructure management, and is more stable than hosting our database locally. We can access our database through our AWS environment or through an SSH tunnel (with verification) improving security and the risk that any unauthorized user can access the database.

Deployment is deployed through Github actions. Each team member contributes their work through the Github repository rather than through a local machine. This workflow makes it accessible to the EC2 server through SSH, which then updates the application, rebuilds the frontend, and publishes the latest files through Nginx. This process is repeatable and reliable which reduces the amount of errors that could potentially happen through manual deployments.

Security

Security is enforced through multiple layers, by network isolation, backend authorization, and secure credential management.

Authentication distinguishes each user as either a student or admin and gives them access control that is enforced through the backend rather than solely relying on the frontend for restrictions. Administrative operations such as modifying course information, degree requirements, and student records are only given to those with Admin permissions.

Database security is hosted through PostgreSQL on Amazon RDS within a private AWS network. The database is not publicly accessible and can only be reached by the application server or through the SSH tunnel for instructor verification. SSH access to the EC2 server uses private key authentication and password based SSH is not available.

Sensitive configuration values, like database credentials and environment variables are stored outside the source code using environment configuration files that are excluded from version control. Passwords are stored as cryptographic hashes instead of plain text to reduce credential compromise. HTTPS is planned for production deployment to protect communication between the users and the application.

Deployment

The student center application uses a cloud based deployment architecture, we are using React/Vite as the frontend that is served through Nginx running on an AWS EC2 Ubuntu server. The Node.js/Express backend communicates with Amazon RDS and PostgreSQL to grab and update students' academic, financial, and enrollment information. We separated the frontend, backend, and database to improve maintainability so that each component is able to improve and progress independently.

Continuous deployment is supported through Github. Source code changes that are pushed through the repository automatically trigger deployment workflow through to the EC2 server, rebuild the application, and publish the updated front end assets. This deployment method allows us to be more consistent without the errors of manual deployment.

Consistency

The student center maintains its consistency by using PostgreSQL as source for all of our information regarding academics, enrollment, and financial information.

Many business rules are enforced by database constraints rather than relying on application logic. Unique constraints prevent duplication of data, for example duplicate enrollment records, duplicate planned courses in the same semester, duplicate class cart entries for a student. Having these constraints guarantee that we have consistent data throughout even if there are multiple requests at a time.

Several user features are generated dynamically to prevent duplication of data. The degree progress tree is computed from Degree programs, requirements, enrollment records, and planned courses ensuring that these changes are made to the student's academic plan immediately. This same process is true for the Calendar which generates class sections, meeting days, enrollment records, and cart selections, eliminating the synchronization issues between stored schedules and displayed schedules.

PostgreSQL transactions, foreign key constraints and relational integrity ensure that all the academic data is consistent between each user, courses, departments, enrollment, and financial information. By centralizing the data within the relational database we minimize redundancy while maintaining accurate relationships throughout the system.

Scalability

Although the initial student center is designed for a small userbase, the architecture would be put in place so that you could expand on this prototype.

Separating the frontend, backend, and database allows for each layer to scale independently as usage increases. Amazon RDS provides us the ability to increase database resources without changing the application and EC2 instances/load balancing can be introduced as more users are introduced to the application.

The database schema is normalized to reduce redundancy while maintaining relationships between all of the academic entities. This improves maintainability and makes querying more efficient as more features get implemented over time.

The architecture separates the generated interface features such as Calendar, Degree progress tree, and campus map from the stored database entities. By computing these views from normalized relational data instead of duplicated information, the system becomes easier to maintain as the student center increases in functionality.

Justification for the design patterns used

The Better Student Center uses creational and structural design patterns to separate responsibilities, simplify complex academic workflows, and reduce dependencies between internal components and external systems.

Singleton Pattern

The database connection pool applies the Singleton pattern. A single shared PostgreSQL connection pool is used by authentication, course search, enrollment, degree planning, billing, and notification services.

Using one shared connection pool prevents the application from creating a new pool for every request. This reduces connection overhead, centralizes database configuration, and helps the application manage a limited number of database connections efficiently.

Factory Method Pattern

The notification service applies the Factory Method pattern to create different notification types, including enrollment notifications, deadline reminders, schedule-conflict warnings, walking-time warnings, billing reminders, and general announcements.

The factory receives the notification type and creates the appropriate notification object. This keeps notification creation logic in one location and allows new notification types to be added without changing every service that produces notifications.

Facade Pattern

The class-cart and enrollment workflow applies the Facade pattern. An Enrollment Facade provides a single interface for adding a class section to the cart or enrolling a student.

Internally, the facade coordinates prerequisite validation, student-hold validation, enrollment-window validation, section-capacity checking, duplicate-selection checking, schedule-overlap detection, and walking-time validation.

This pattern is appropriate because enrollment requires several related checks. The facade prevents controllers and frontend components from coordinating each validator separately and provides one consistent entry point for the complete workflow.

Adapter Pattern

The system applies the Adapter pattern when communicating with external services such as institutional authentication, payment processing, campus maps, and external academic systems.

Each adapter converts the external system's request and response formats into interfaces understood by the Better Student Center. This isolates external dependencies from the application's business logic. If an external provider changes its interface, only the corresponding adapter needs to be updated.

Layered Architecture

The backend follows a layered architecture. API controllers receive and validate HTTP requests, business services implement academic and enrollment rules, repositories manage database operations, and PostgreSQL provides persistent data storage.

This separation allows presentation, business logic, and data access components to be maintained and tested independently. It also prevents database queries and complex academic rules from being placed directly inside API controllers.

Communication between the frontend and backend follows RESTful API principles. Standard HTTP methods expose resources and services related to users, courses, enrollment, degree progress, schedules, billing, and notifications.

The database follows a normalized relational design that minimizes duplicated data and preserves relational integrity. Dynamic features such as degree progress, weekly calendars, and the campus map are generated from shared academic records rather than stored as duplicate data.

Incremental Software Process

The Better Student Center is developed using an incremental software process. Each increment delivers a complete vertical feature that connects the frontend, backend, and database and can be integrated and tested independently.

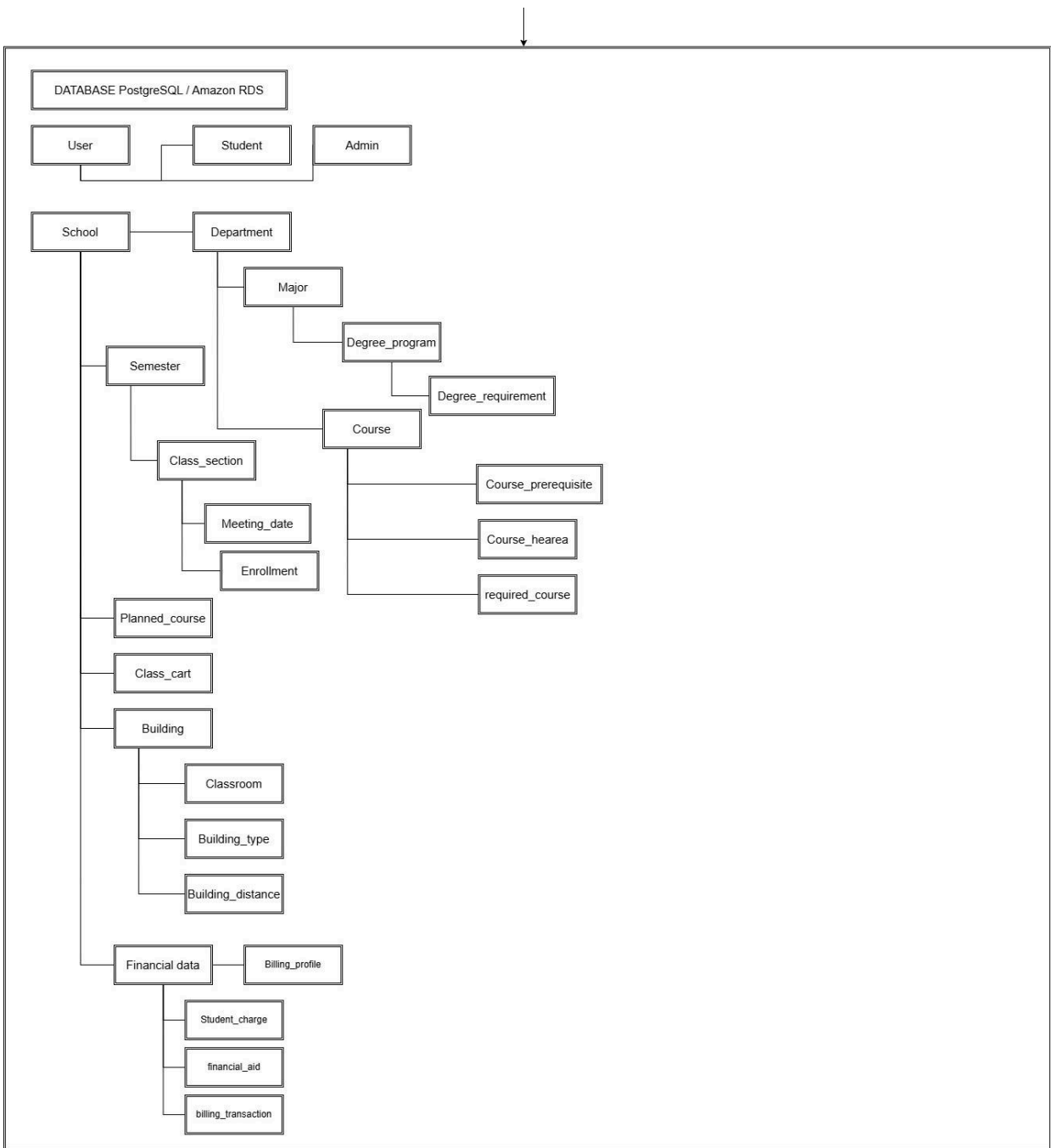
The increments are organized around major functional areas, including authentication and course search, class-cart and enrollment validation, degree planning and schedule management, and billing and notification services. Each functional increment includes its user interface, API behavior, business rules, database operations, and system testing.

After an increment is integrated, the team evaluates its functionality, architecture, and interaction with existing features before introducing additional functionality. Feedback and testing results from one increment can therefore guide the design of later increments.

This process reduces integration risk, allows the team to identify architectural problems early, and ensures that the system grows through complete working features rather than through disconnected frontend, backend, and database components.

UML Diagram





Network and Deployment Design

The Better Student Center prototype is deployed using Amazon Web services. The current deployment uses one AWS EC2 Ubuntu server as the application server. The EC2 instance hosts the frontend environment through Nginx and supports the planned [Node.js/Express](#) backend service for API requests.

The public website is available at: <http://18.226.97.234>

The React/Vite frontend build is served by Nginx from: /var/www/html

The project source code is stored in the team GitHub repository. GitHub Actions is used to support deployment by connecting to the EC2 server through SSH, pulling the latest project code, building the frontend, and updating the public frontend directly.

The PostgreSQL database is hosted separately using Amazon RDS,

The RDS endpoint is: **student-center-db.chwwk2s88tac.us-east-2.rds.amazonaws.com**

The current database used for verification is: postgres

Amazon RDS PostgreSQL was selected because the project stores structured relational data such as users, students, departments, courses, class sections, planned courses, building, classrooms, enrollment records, degree requirement, and financial records. The M2 database architecture also explains that PostgreSQL supports primary keys, foreign keys, constraints, transactions, and relational queries, which are needed for consistent academic and enrollment data.

The RDS database is private and is not intended for direct public access. Authorized users can access the database from the EC2 server or through an SSH tunnel using localhost:5433.

Current Network and Deployment infrastructure

Student / Admin Browser

|

| HTTP port 80

v

AWS EC2 Ubuntu Server

Public IP: 18.226.97.234

Private IP: 172.31.41.182

|

| Nginx serves React/Vite build

v

Public Website

http://18.226.97.234

Developer Computer

|

| git push

v

GitHub Repository

|

| GitHub Actions SSH deployment

v

AWS EC2 Ubuntu Server

|

| Build React/Vite frontend

| Copy production files to /var/www/html

v

Nginx serves updated frontend

AWS EC2 Ubuntu Server

|

| PostgreSQL port 5432

| Private AWS network connection

v

Amazon RDS PostgreSQL

Endpoint: student-center-db.chwwk2s88tac.us-east-2.rds.amazonaws.com

Database: postgres

Roles: Jeff, instructor_verifier, team_developer

Instructor Verification Computer

|

| SSH port 22 using bigbacks-professor-access.pem

v

AWS EC2 Ubuntu Server

|

| SSH tunnel localhost:5433

v

Amazon RDS PostgreSQL

Deployment Components

Component	Current Use in Project
GitHub Repository	Stores the team source code and project files
GitHub Actions	Supports deployment to the EC2 server through SSH
AWS EC2 Ubuntu Server	Hosts the deployed application environment
Nginx	Serves the React/Vite frontend from /var/www/html
React/Vite Frontend	Provides the public user interface for Student and Admin users
Node.js/Express Backend	Handles API requests and database communication
Amazon RDS PostgreSQL	Hosts the private PostgreSQL database
Credentials Folder	Provides instructor access instructions and the required professor PEM file

Network Protocols and Ports

Connection	Protocol / Port	Purpose
Student/Admin browser to EC2	HTTP / 80	Allows users to access the public frontend
Student/Admin browser to EC2	HTTPS / 443	Planned secure browser access
Developer or GitHub Actions to EC2	SSH / 22	Used for deployment and server administration
EC2 to RDS PostgreSQL	PostgreSQL / 5432	Allows the backend/application server to connect to the database
Local computer to RDS through SSH tunnel	localhost / 5433	Allows authorized reviewers to verify the private database through EC2

Security Notes

The RDS PostgreSQL database is private and should not be opened directly to the public internet. Database verification should be performed from the EC2 server or through an SSH tunnel. SSH access to EC2 uses private-key authentication. Database passwords, .env file, JWT secrets, and personal private keys should not be committed to GitHub.

High Level APIs and Algorithms

Major Application Services

1. Authentication and User Service
 - Provides student registration, authentication, authorization, and profile management while protecting credentials and enforcing role-based access.
2. Course Search Service
 - Allows students to search the course catalog using subject code, course number, title, or description. Results combine course and section information, including meeting days, modality, capacity, enrollment count, and available seats.
3. Degree Planning and Progress
 - Compares completed, enrolled, and planned courses with degree-program requirements. It identifies completed, in-progress, planned, and missing requirements and organizes them into major, elective, general-education, and university requirement categories.
4. Class Cart and Enrollment Service
 - Coordinates class selection and enrollment validation. It checks duplicate selections, prerequisites, holds, enrollment windows, course capacity, schedule overlap, and travel time between classrooms before accepting a class.
5. Weekly Calendar and Schedule Validation Service
 - Generates a weekly calendar from enrolled and cart sections. It groups meetings by weekday, orders them by start time, and identifies overlapping classes or insufficient walking time between campus buildings.
6. Billing Service
 - Calculates the student's current balance by combining charges, completed payments, refunds, and financial-aid records. It also provides due dates and transaction history.
7. Notification Service
 - Creates and retrieves enrollment, deadline, schedule-conflict, walking-time, billing, and general notifications.

Endpoint	Method	Description
/api/courses	GET	Searches across subject code, course number, title, and description; combines course and section data and calculates available seats.
/api/cart/sections	POST	Validates duplicates, prerequisites, holds, enrollment windows, capacity, schedule overlap, and walking time before adding a section.

/api/degree-progress	GET	Compares completed, enrolled, and planned courses with degree requirements and returns a categorized progress tree.
/api/calendar	GET	Builds a weekly schedule, orders meetings by time, and reports time-overlap and walking-time conflicts.
/api/billing/balance	GET	Aggregates charges, completed payments, refunds, and financial aid to calculate the current balance.

Internal Backend APIs

1. React

- Purpose: Builds an interactive user interface using reusable components.
- Used in: Login, Course Search, Degree Planner, Calendar, Billing, Campus Map, Student Profile, and Administrative dashboard

2. Express.js

- Purpose: Provides the backend service layer between the React frontend and PostgreSQL database..
- Used in: Authentication, enrollment validation, degree planning, notifications, billing, and administrator management

3. Node-postgres

- Purpose: Enables secure communication between the backend Express and PostgreSQL.
- Used in: All database queries for users, courses, enrollment, billing, planner, and notifications

4. Nginx

- Purpose: Serves the React frontend and proxies API requests to the Express backend.
- Used in: Production deployment on AWS EC2

5. Github Actions

- Purpose: Automates application integration, build, and deployment to the AWS EC2 server.
- Used in: Continuous Integration and Continuous Deployment

6. Amazon Web Services

- Purpose: Hosts the production infrastructure for the application
- Used in: EC2 - application server, Amazon RDS - PostgreSQL database, Security groups - network access control

Non-trivial Algorithms

1) Class Cart Validation Algorithm

Validates whether a student can add a class section to their enrollment cart.

Process

1. Retrieve the selected class section and the student's academic record.
2. Confirm that the class section exists and has not been cancelled.
3. Check whether the class section is already present in the student's cart or enrollment record.
4. Retrieve the course prerequisites and compare them with the student's completed courses and grades.
5. Check whether the student has an active hold that blocks enrollment.
6. Verify that the applicable enrollment window is open.
7. Compare the section capacity with its current enrollment count to determine seat availability.
8. Retrieve the student's enrolled and cart sections.
9. Compare meeting days and time intervals to identify schedule overlaps.
10. For consecutive in-person classes, calculate the required travel time using the distance between buildings and the student's walking speed.
11. Return any validation errors to the frontend.
12. Add the class section to the cart only when no blocking condition is found.

2) Degree Progress Tree Generation Algorithm

Builds the students' degree progress tree dynamically.

Process

1. Retrieve the students active degree program
2. Retrieve all degree requirements for that program
3. Retrieve completed courses, enrolled courses, and planned courses
4. Compare each requirement against the student's academic record
5. Mark each requirement as Completed, In-Progress, Planned, or Missing
6. Group requirements by category, such as core courses, electives, and general education
7. Return the structured progress tree to the front end

3) Weekly Calendar Generation Algorithm

Creates the student's weekly calendar from schedule data.

Process

1. Retrieve enrolled class sections for the student
2. Retrieve class cart section for schedule preview
3. Retrieve meeting days, start times, end times, classrooms, and buildings
4. Convert each class meeting into a calendar event
5. Group calendar events by weekday
6. Sort each days event by start time
7. Return the weekly calendar structure to the frontend

4) Billing Balance Calculation Algorithm

Calculates the student's current financial balance.

Process

1. Retrieve the students billing profile
2. Retrieve tuition charges, fees, and student charges

3. Retrieve financial aid records
4. Retrieve completed billing transactions
5. Add all charges
6. Subtract payments and financial aid
7. Return the remaining balance and transaction summary

Key Project Risks

1. External System Dependencies

A lot of our planned features rely on systems we don't control, including SFSU SSO, Canvas, the financial aid portal, and the payment portal. We do not currently have confirmed API access to any of these services, so if they are unavailable or behave differently than expected, features such as login, Canvas redirects, and payment processing may not work correctly.

This risk has a high impact because authentication is required before users can access most other features. To reduce this risk, we plan to build mock versions of these services so development can continue even if access to the real systems is delayed.

2. Degree Progress Tree and Prerequisite Logic

The Degree Progress Tree is one of the most technically challenging parts of the project. The system must track completed, planned, and remaining requirements across General Education categories, major requirements, and electives. It must also update dynamically when a student changes their schedule or declares a major.

If this logic is incorrect, students could receive inaccurate academic planning information. To reduce this risk, we will build and test the feature using realistic student scenarios and compare results against official SFSU catalog requirements.

3. Database Schema Stability

The project currently contains more than twenty functional requirements with many relationships between them. Changes to core requirements such as Course, Enrollment, or Student could affect multiple parts of the application and require significant rework.

To minimize this risk, we plan to finalize the core database design and ERD during Milestone 2. Database migrations will be used from the beginning of development so schema changes are tracked and can be rolled back if necessary.

4. Security

The application handles academic and financial information that should only be accessible to authorized users. Improper access control could allow students to view data that does not belong to them or perform actions they should not have permission to perform.

To reduce this risk, role-based access control will be enforced on the backend rather than relying solely on frontend restrictions. Secure authentication, HTTPS communication, and authorization checks on protected endpoints will also be implemented.

5. Team Integration

Development work is currently divided among frontend, backend, and database responsibilities. Since different components are being developed separately, integration issues may appear when features are combined.

In previous milestones, some assumptions about interfaces between components resulted in rework. To address this, all code changes will go through pull requests and at least one full integration test will be completed during each milestone.

6. Scope and Time Constraints

The project includes a large number of planned features relative to the available development time. Features such as the campus map and walking-time calculations add complexity but are not as critical as enrollment and degree planning functionality.

To manage this risk, the team will prioritize core academic planning and enrollment features first. Lower-priority features will be treated as stretch goals if time allows.

External Dependencies

The Student Center application relies on several external systems and services during development and deployment.

- **AWS EC2 (Ubuntu):** Hosts both the React frontend and Express backend. The application runs on EC2 instances and depends on AWS for uptime and availability.
- **Amazon RDS PostgreSQL:** Manages the application database. RDS handles backups and is only accessible within the AWS environment, not from the public internet.
- **GitHub Actions:** Used for continuous integration and deployment. Code pushed to the repository can trigger automated workflows for testing and deployment.
- **Nginx:** Acts as a reverse proxy on the EC2 instance, routing incoming requests to the Express backend and serving the React frontend.
- **Canvas:** Provides access to course pages for enrolled students.
- **Financial Aid Website:** Used for financial aid applications and award information.
- **University Payment Portal:** Used for processing student payments and viewing outstanding balances.

Security Discussion

Since the Student Center application stores academic and financial information, security is an important part of the project.

Access to the EC2 instances is restricted to SSH key authentication. Password-based SSH login is disabled. AWS security groups are configured to limit which ports and IP addresses can reach the servers, and the Amazon RDS database is not publicly accessible — it can only be reached from within the AWS environment.

Database access is managed through dedicated database users with limited permissions rather than using a root or admin account for application queries. Passwords stored in the database will be hashed and never stored in plain text.

The application will use role-based access control, allowing students to access student features and admins to access administrative functions. Access checks are enforced on the backend rather than relying solely on the frontend.

Sensitive information such as database passwords, private keys, and environment variables are stored in .env files that are excluded from version control. These values are never committed to the GitHub repository. HTTPS will be used for all communication between the client and the server to protect data in transit.

Project Management

For milestone 2, the team managed the work by dividing the document and prototype responsibilities based on each member's project and technical strength. Since the milestone includes documentation, database design, backend planning, frontend mockups. Development planning, risks, and project management, the team separated the work into similar sections so that each member could focus on a specific area.

Wen-Chien Yen, as the Team Lead and Scrum Master, coordinated the milestone 2 work by reviewing assignment requirements, identifying the major sections that needed to be completed, and assigning responsibilities to each team member. The work was divided across data definitions, prioritized functional requirements, mockups and storyboards, database architecture, backend architecture, network and deployment design, high-level APIs and algorithms, key project risks, external dependencies, security discussion, and team contributions.

Team members communicated through group messages and shared document editing. The Google Doc was used as the main collaboration space for writing and reviewing Milestone 2 content. Team members were responsible for adding their assigned parts directly into the shared document. The team lead reviewed the document structure. Checked whether required sections were missing, and helped make sure the content matched the current project scope.

For milestone 2, the team also used GitHub to manage project files and code. The deployed prototype is connected to the team's GitHub repository, and GitHub Actions is used to support deployment to AWS EC2 server. This helps the team keep the source code organized and makes deployment easier when frontend updates are pushed.

Team Member	Milestone 2 Responsibility
Wen Chien Yen	Team lead coordination, network and deployment design, AWS EC2/RDS setup explanation, credentials verification, project management, final integration, and submission review
Oscar Wang	Frontend mockups, storyboards, and React frontend interface planning
Banghao Yuan	Prioritized functional requirements, use case alignment, and requirement testing support
Sean Wang	Data definitions, database architecture, PostgreSQL/RDS explanation, and ERD support
Enoch Lin	Backend architecture, API planning, algorithms, and GitHub/backend support
Kashif Zada	Key project risks, external dependencies, security discussion, scalability planning, and backend support

For future milestones, the team will continue using a similar task-based workflow, but with more emphasis on implementation and integration. Instead of only dividing documentation sections, future work will be divided by working features. For example, one member may work on the frontend page, another member may work on the backend API, another member may work on the database tables, and another member may test whether the full feature works from the browser to the database.

Future milestones will be managed by setting smaller goals for each checkpoint. The team will prioritize core prototype features first, including login, course search, planned schedule, database connection, and frontend-backend integration. Lower-priority features, such as advanced map behavior, financial information, and external redirects, will be handled after the core academic planning features are working.

The team will also improve integration testing in future milestones. Since the project has frontend, backend, database, and deployment parts, each completed feature should be tested across the full stack before submission. This means checking that the React frontend

sends the correct request, the Express backend handles the request, PostgreSQL stores or returns the correct data, and the deployed public website still works after changes.

Team Contributions

Team lead assigned contribution scores. All team members have submitted confidential feedback via the instructor provided Google form.

Team Member	Wen Chien Yen	Oscar Wang	Banghao Yuan	Sean Wang	Enoch Lin	Kashif Zada
Contribution Score	10	10	10	10	10	10

Team Contributions

Wen-Chien Yen

I contributed to Milestone 2 as the Team Lead and Scrum Master. I worked on the Network and Deployment Design section by documenting the current AWS deployment setup, including the EC2 Ubuntu server, Nginx, React/Vite frontend deployment, GitHub Actions SSH deployment flow, private Amazon RDS PostgreSQL access, and instructor verification tunnel. This work helped explain how the project is deployed and how authorized reviewers can access the cloud infrastructure.

I also completed the Project Management section. It explained how Milestone 2 work was divided among team members, how the team used GitHub and shared documentation, and how future milestones will be managed through feature-based implementation and integration testing. As team lead, I also helped coordinate task assignments, monitor progress, review required milestone sections, and support the final integration of documentation and deployment-related materials.

Oscar Wang

Oscar Wang contributed to the frontend structure for the Student Center project. He created the skeleton base for the website navigation and helped organize the frontend structure so that the application can support page transitions and future feature pages. He also restructured several frontend components and entities to make the code cleaner and easier to maintain.

Oscar also added an initial template for the login page. His work helped prepare the frontend for the vertical prototype by creating a foundation for user navigation, authentication-related pages, and future frontend integration with backend APIs.

Oscar also designed the mockups for the application, as well as wrote the specific storyboards that followed our general use cases.

Banghao Yuan

Banghao Yuan contributed to both the documentation and implementation sides of Milestone 2. For the documentation, he worked on the Data Definitions and Prioritized High Level Functional Requirements sections. He added data definitions for core Student Center entities, including users, students, courses, class sections, schedules, campus map data, notifications, and financial records. He also clarified key attributes and relationships for these entities.

Banghao also organized and refined the high-level functional requirements based on Milestone 1 Version 2. He grouped requirements into Priority 1, Priority 2, Priority 3, Maybe Later, and Out of Scope categories to clarify the current prototype scope and future development priorities.

For Backend Architecture, Banghao created the initial UML diagram and mapped frontend modules to REST API endpoints, backend controllers, PostgreSQL/RDS database entities, and data flow. He also helped draft the first version of the API design. Later, he connected the backend with the database, prepared API endpoints for frontend-backend communication, and implemented functional requirements for user registration and course search.

Sean Wang

Sean Wang contributed to the Database Architecture section and database implementation for Milestone 2. He finalized the list of functional requirements for M2v1, updated the ERD with cardinality, and created the PostgreSQL schema based on the Milestone 2 design. He also added an ERD based on the database design, helping the team clearly represent the database entities and their relationships.

Sean also created `seed.sql` to populate several database tables, especially tables related to course search. In addition, he worked on connecting the frontend with the backend by creating a course search page using `courses.js` from Banghao. The course search feature uses `ORDER BY` sorting to organize course items, supporting the vertical prototype requirement for functional search.

Enoch Lin

Enoch Lin contributed as the GitHub Master and supported the team's backend and repository workflow. His role helped the team organize code changes and maintain project collaboration through GitHub. Although several backend architecture tasks were completed by Banghao and Sean, Enoch's GitHub role supported the team's ability to coordinate branches, manage code updates, and prepare the project for integration.

For Milestone 2, Enoch's contribution should continue to focus on GitHub organization, backend support, and helping ensure that code related to the vertical prototype is committed, reviewed, and merged correctly into the default branch for grading.

Kashif Zada

Kashif Zada contributed to the Key Project Risks, External Dependencies, and Security Discussion sections. He wrote the Key Project Risks section covering external system dependencies, degree progress logic, database schema stability, security, team integration, and scope/time constraints. These risks helped the team identify possible technical and management problems before later milestones.

Kashif also documented the project's external dependencies, including AWS EC2, Amazon RDS PostgreSQL, Nginx, GitHub Actions, SFSU SSO, Canvas, financial aid, and payment portals. In the Security Discussion section, he covered SSH key access, security groups, role-based access control, password hashing, HTTPS, and secrets management. His work helped explain how the project depends on external systems and how the team plans to protect infrastructure, credentials, and user data.